UNIVERSIDADE DO VALE DO ITAJAÍ
ESCOLA POLITÉCNICA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ANÁLISE DA OTIMIZAÇÃO DE CNNS 1D NA CLASSIFICAÇÃO DE IMAGENS HIPERESPECTRAIS

por

Eduardo Savian

Itajaí (SC), Dezembro de 2026

UNIVERSIDADE DO VALE DO ITAJAÍ
ESCOLA POLITÉCNICA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ANÁLISE DA OTIMIZAÇÃO DE CNNS 1D NA CLASSIFICAÇÃO DE IMAGENS HIPERESPECTRAIS

Área de Visão Computacional

por

Eduardo Savian

Relatório apresentado à Banca Examinadora do Trabalho Técnico-científico de Conclusão de Curso de Ciência de Computação para análise e aprovação.
Orientador(a): Felipe Viel, PhD.

Itajaí (SC), Dezembro de 2026

Todos os Homens, por natureza, desejam conhecer - Aristóteles

RESUMO

A classificação de imagens hiperespectrais (HSI) com redes neurais convolucionais unidimensionais (CNNs 1D) é uma alternativa leve às arquiteturas espaciais, e a fusão do identificador do superpixel às características espectrais compensa parte da falta de contexto espacial. Este trabalho parte da CNN 1D com fusão SEEDS de Souza (2023) e a otimiza em quatro eixos: organização do código, algoritmos de segmentação, otimização de hiperparâmetros e paralelismo, medindo cada ganho contra a configuração original. O fluxo foi reimplementado em PyTorch como um pacote modular e determinístico, com os dados idênticos aos do original e a CNN 1D do projeto, com *max-pooling*. A avaliação usa Indian Pines, University of Pavia e Salinas Valley. O LSC e o Watershed foram calibrados para gerar o mesmo número de regiões que o SEEDS; a busca em grade foi comparada à otimização bayesiana com processo gaussiano; e o paralelismo foi implementado na partição da cena em faixas, em threads ou processos, e na fusão concorrente, em que uma thread segmenta enquanto as demais calculam as convoluções. Com os hiperparâmetros do original, a CNN do projeto obteve OA de 0,9142 em Indian Pines, contra 0,8960 da CNN original. Com 5 sementes, o SEEDS superou o LSC nas três cenas e empatou com o Watershed, que segmenta de 28 a 62 vezes mais rápido. Sobre a configuração original, a busca acrescentou 2,55 pontos percentuais de OA com o SEEDS e 3,25 com o Watershed, acima do dobro do desvio combinado. Grade e bayesiana empataram com o mesmo orçamento de 27 *trials*. A partição em processos já abertos chegou a 6,06 vezes a execução sequencial com 16 faixas, enquanto as threads perderam para ela a partir de 4; a fusão concorrente chegou a 1,2 a 1,4 vez a execução em série, perto do teto dado pela segmentação.

Palavras-Chave: Imagem Hiperespectral. Rede Neural Convolucional Unidimensional. Segmentação. Otimização de Hiperparâmetros. Processamento Paralelo.

ABSTRACT

Hyperspectral image (HSI) classification with one-dimensional convolutional neural networks (1D CNNs) is a lightweight alternative to spatial architectures, and fusing the superpixel identifier with the spectral features compensates for part of the missing spatial context. This work starts from the 1D CNN with SEEDS fusion of Souza (2023) and optimizes it along four axes: code organization, segmentation algorithms, hyperparameter optimization, and parallelism, measuring each gain against the original configuration. The pipeline was reimplemented in PyTorch as a modular, deterministic package, with the same input data as the original and the project's 1D CNN, with max pooling. The evaluation uses Indian Pines, University of Pavia, and Salinas Valley. LSC and Watershed were calibrated to produce the same number of regions as SEEDS; grid search was compared with Bayesian optimization with a Gaussian process; and parallelism was implemented as scene partitioning into strips, in threads or processes, and as concurrent fusion, in which one thread segments while the others compute the convolutions. With the original hyperparameters, the project CNN reached an OA of 0.9142 on Indian Pines, against 0.8960 for the original CNN. With 5 seeds, SEEDS outperformed LSC on all three scenes and tied with Watershed, which segments 28 to 62 times faster. Over the original configuration, the search added 2.55 percentage points of OA with SEEDS and 3.25 with Watershed, above twice the combined deviation. Grid and Bayesian search tied with the same budget of 27 trials. Scene partitioning in already started processes reached 6.06 times the sequential speed with 16 strips, while threads were slower from 4 strips on; concurrent fusion reached 1.2 to 1.4 times the serial speed, close to the ceiling set by segmentation.

***Keywords**: Hyperspectral Image. One-Dimensional Convolutional Neural Network. Segmentation. Hyperparameter Optimization. Parallel Processing.*

LISTA DE ILUSTRAÇÕES

Figura 1 – Imagens de satélite LANDSAT da área de Washington D.C. em diferentes bandas espectrais temáticas 20
Figura 2 – Estrutura de dados de uma imagem hiperespectral: (a) aquisição por sensor em movimento; (b) formação do hipercubo tridimensional; (c) dimensões espaciais e canais espectrais 21
Figura 3 – Exemplo de operação de erosão morfológica aplicada a uma imagem binária usando elementos estruturantes de diferentes tamanhos 23
Figura 4 – Exemplo de processamento de nível médio: segmentação de células em imagens microscópicas 23
Figura 5 – Exemplo de processamento de alto nível: correspondência de características usando SIFT, mostrando 41 pares encontrados com apenas 3 erros reais . . . 24
Figura 6 – Tipos de mapa de segmentação: bordas (superior) e regiões (inferior) gerados pelo algoritmo SEEDS 25
Figura 7 – Diagrama simplificado mostrando como as avaliações são respaldadas através da árvore de movimentos possíveis para chegar ao melhor próximo movimento 28
Figura 8 – Comportamento da descida de gradiente durante o treinamento de uma rede convolucional: (Esquerda) distribuição das normas do gradiente ao longo do tempo; (Direita) taxa de erro de classificação no conjunto de validação . . . 29
Figura 9 – Paradigmas de aprendizagem e sua relação com os tipos de dados de treinamento 29
Figura 10 – Redes neurais multicamadas e backpropagation: (a) transformação do espaço de entrada; (b) regra da cadeia; (c) equações do forward pass; (d) equações do backward pass 31
Figura 11 – Arquitetura de uma rede neural convolucional mostrando camadas de convolução com ReLU e max pooling 31
Figura 12 – Operação de convolução mostrando a aplicação do kernel sobre a matriz de entrada 33
Figura 13 – Arquitetura CNN 1D para classificação de imagens hiperespectrais: pré-processamento PCA seguido de 4 camadas convolucionais 1D e camadas densas para classificação 35
Figura 14 – Quatro threads de um processo executando código Python e código nativo que libera o GIL 38
Figura 15 – Comparação entre o tamanho global do lote e o número de GPUs por *trial* no ASHA e no RASDA, nos diferentes casos de aplicação 42
Figura 16 – Curvas de acurácia de treino e de validação do TensorFlow com as estratégias sequencial e de paralelismo de dados no CIFAR-10 43
Figura 17 – Curvas de acurácia de treino e de validação do PyTorch com as estratégias sequencial e de paralelismo de dados no CIFAR-10 43

Figura 18 – Fluxograma da metodologia proposta, com os ramos espacial e espectral, a fusão via concatenação, a classificação e a validação iterativa do modelo . . 44
Figura 19 – Principais componentes do projeto 47
Figura 20 – Indian Pines: composição falsa-cor, mapa de verdade terrestre e legenda . . 49
Figura 21 – Salinas Valley: (a) falsa-cor, (b) verdade terrestre, (c) legenda 50
Figura 22 – University of Pavia: (a) falsa-cor, (b) verdade terrestre e legenda 51
Figura 23 – Arquitetura modular do sistema 53
Figura 24 – Arquitetura da CNN 1D prevista no projeto 54
Figura 25 – Processo de fusão por concatenação 56
Figura 26 – Estratégia de paralelização: fluxo sequencial versus fluxo paralelo 57
Figura 27 – Fluxo da otimização de hiperparâmetros 58
Figura 28 – Composição RGB de Indian Pines e as regiões de cada segmentador, com os parâmetros padrão 64
Figura 29 – Um lote processado pixel a pixel e na GPU 66
Figura 30 – Os ramos do pré-processamento em sequência e em duas threads 66
Figura 31 – A cena em quatro faixas e o treino delas em cada executor 67
Figura 32 – Inferência em lote e fusão concorrente 68
Figura 33 – OA de teste com a CNN otimizada para cada segmentador (experimento 03) e tempo de uma execução em Indian Pines 72
Figura 34 – OA de teste das configurações original e otimizada (experimento 05) e maior validação até cada *trial* com até 200 épocas (experimento 04), em Indian Pines 74
Figura 35 – *Speedup* do modo independente sobre a execução sequencial (experimento 07) e OA com um modelo por faixa (experimento 06), em Indian Pines . . . 75
Figura 36 – Fusão concorrente contra a execução em série (experimento 08) 76

LISTA DE TABELAS

Tabela 1 – Camadas da CNN 1D do projeto com fusão, em Indian Pines ($K_b = 30$) . . 65
Tabela 2 – Metas do projeto, valores reportados por Souza (2023) e valores do experimento 01 70
Tabela 3 – Estrutura das segmentações com os parâmetros padrão 72
Tabela 4 – Segmentadores com a CNN fixa (experimento 02) e com a CNN otimizada para cada um (experimento 03) 73
Tabela 5 – Configurações vencedoras da busca do experimento 03 73
Tabela 6 – Grade e Otimização Bayesiana com até 200 épocas por *trial* (experimento 04) 74

LISTA DE QUADROS

Quadro 1. Comparação dos trabalhos relacionados 46
Quadro 2. Requisitos funcionais 52
Quadro 3. Requisitos não funcionais 52
Quadro 4. Descrição detalhada dos experimentos 59
Quadro 5. Planejamento de experimentos 59
Quadro 6. Plano de verificação dos requisitos funcionais 60
Quadro 7. Plano de verificação dos requisitos não funcionais 61
Quadro 8. Validação de desempenho nos datasets de referência 62
Quadro 9. Análise de riscos 62
Quadro 10. Espaço de busca de hiperparâmetros 68
Quadro 11. Experimentos da campanha 69
Quadro 12. Evidência de cada requisito 77

LISTA DE EQUAÇÕES

Equação 1. Projeção PCA 22
Equação 2. Reconstrução Geodésica Morfológica 25
Equação 3. Vetor de Derivada do Perfil Morfológico (DMP) 25
Equação 4. Função de Energia do Algoritmo SEEDS 26
Equação 5. Acurácia Geral (OA) 26
Equação 6. Acurácia Média (AA) 27
Equação 7. Coeficiente Kappa 27
Equação 8. Função de Ativação ReLU 30
Equação 9. Gradiente da Função de Perda 31
Equação 10. Entropia Cruzada 32
Equação 11. Operação de Convolução 1D 32
Equação 12. Entrada Líquida em Camada Convolucional 33
Equação 14. Lei de Amdahl 38
Equação 15. Teto do Paralelismo entre Duas Etapas 39
Equação 16. Melhora Esperada 39

LISTA DE ABREVIATURAS E SIGLAS

1D	Unidimensional (*One-Dimensional*)
AA	Acurácia Média (*Average Accuracy*)
AM	Manufatura Aditiva (*Additive Manufacturing*)
ASHA	Asynchronous Successive Halving Algorithm
CFD	Fluidodinâmica Computacional (*Computational Fluid Dynamics*)
CNN	Rede Neural Convolucional (*Convolutional Neural Network*)
CPU	Unidade Central de Processamento (*Central Processing Unit*)
CUDA	Arquitetura de Dispositivo de Computação Unificada (*Compute Unified Device Architecture*)
DL	Aprendizado Profundo (*Deep Learning*)
DMP	Derivada do Perfil Morfológico (*Derivative Morphological Profile*)
FMA	Multiplicação e Soma Fusionadas (*Fused Multiply-Add*)
GIL	Trava Global do Interpretador (*Global Interpreter Lock*)
GPU	Unidade de Processamento Gráfico (*Graphics Processing Unit*)
GT	Ground Truth
HPC	Computação de Alto Desempenho (*High Performance Computing*)
HPO	Otimização de Hiperparâmetros (*Hyperparameter Optimization*)
HSI	Imagem Hiperespectral (*Hyperspectral Imaging*)
IA	Inteligência Artificial
IP	Indian Pines
LSC	Linear Spectral Clustering
ML	Aprendizado de Máquina (*Machine Learning*)
MLP	Perceptron Multicamadas (*Multilayer Perceptron*)
MS	Mapa de Segmentação
MSE	Erro Quadrático Médio (*Mean Squared Error*)

OA	Acurácia Geral (*Overall Accuracy*)
OF	Sobreajuste (*Overfitting*)
PCA	Análise de Componentes Principais (*Principal Component Analysis*)
RAM	Memória de Acesso Aleatório (*Random Access Memory*)
RASDA	Resource-Adaptive Successive Doubling Algorithm
ReLU	Unidade Linear Retificada (*Rectified Linear Unit*)
RGB	Vermelho, Verde e Azul (*Red, Green, Blue*)
SEEDS	Superpixels Extracted via Energy-Driven Sampling
SGD	Descida de Gradiente Estocástica (*Stochastic Gradient Descent*)
SIMT	Instrução Única, Múltiplas Threads (*Single Instruction, Multiple Threads*)
SM	Multiprocessador de *Streaming* (*Streaming Multiprocessor*)
SSL	Aprendizado Semissupervisionado (*Semi-supervised Learning*)
SV	Salinas Valley
UP	University of Pavia
WS	Watershed

LISTA DE SÍMBOLOS

κ	Coeficiente Kappa
E	Experiência (no contexto de aprendizado de máquina)
L	Função de perda (*Loss function*)
P	Medida de desempenho
T	Classe de tarefas
θ	Parâmetros do modelo
∇	Operador gradiente
GB	Gigabyte
MB	Megabyte
ms	Milissegundo
s	Segundo
TB	Terabyte

SUMÁRIO

1 INTRODUÇÃO 17
1.1 Problematização 18
1.1.1 Solução Proposta 18
1.2 Objetivos 18
1.2.1 Objetivo Geral 18
1.2.2 Objetivos Específicos 18
1.3 Metodologia 19
1.4 Estrutura do Trabalho 19

2 FUNDAMENTAÇÃO TEÓRICA 20
2.1 Imagem Hiperespectral 20
2.1.1 Redução de Dimensionalidade 21
2.2 Processamento de Imagens 22
2.2.1 Segmentação de Imagens 24
2.2.2 Métricas de Avaliação 26
2.3 Aprendizado de Máquina 27
2.3.1 Desafios Centrais: Generalização e Sobreajuste 28
2.3.2 Paradigmas de Aprendizagem 29
2.3.3 Aprendizado Profundo 30
2.3.3.1 Redes Neurais Convolucionais 32
2.3.3.2 Aplicações de CNNs 34
2.4 Processamento Paralelo em GPUs 35
2.4.1 Arquitetura de GPUs e CUDA Cores 35
2.4.2 Paralelização de Operações de Convolução 36
2.4.3 Estratégias de Paralelização para o Fluxo de Classificação 36
2.5 Concorrência em CPU 37
2.5.1 Threads, Processos e o GIL 37
2.5.2 Exclusão Mútua e Variáveis de Condição 38
2.5.3 Lei de Amdahl e *Speedup* 38
2.6 Otimização de Hiperparâmetros 39
2.7 Trabalhos Relacionados 40
2.7.1 Protocolo de Análise 40
2.7.2 Parallel and Scalable Hyperparameter Optimization for Distributed Deep Learning Methods on High-Performance Computing Systems 41
2.7.3 Comparative Analysis of TensorFlow and PyTorch for Image Classification using CNN with Parallelism Strategies 41
2.7.4 Avaliação de Fusão de Dados Espaciais com Espectrais para Classificação de Imagens Hiperespectrais 43

2.7.5 Análise Comparativa dos Trabalhos Relacionados 45

3 PROJETO 47
3.1 Visão Geral 47
3.2 Premissas 48
3.3 Metodologia 48
3.3.1 Materiais 48
3.3.1.1 Ambiente Computacional e Ferramentas 48
3.3.1.2 Datasets Hiperespectrais 49
3.3.2 Métodos 51
3.4 Análise de Requisitos 52
3.4.1 Arquitetura do Sistema 52
3.4.2 Algoritmos e Técnicas 54
3.4.2.1 Redução Dimensional via PCA 54
3.4.2.2 Rede Neural Convolucional 1D (CNN 1D) 54
3.4.2.3 Algoritmos de Segmentação Espacial 55
3.4.2.4 Fusão de Características por Concatenação 55
3.4.2.5 Paralelização do Processamento 56
3.4.2.6 Otimização Automática de Hiperparâmetros 57
3.5 Plano de Verificação 58
3.5.1 Campanha Experimental 58
3.5.2 Análise Estatística dos Resultados 60
3.5.3 Validação dos Requisitos 60
3.6 Análise de Riscos 62

4 DESENVOLVIMENTO 63
4.1 Ambiente e Organização do Código 63
4.2 Fluxo de Execução 63
4.2.1 Segmentadores 64
4.2.2 Modelo e Fusão 64
4.2.3 Treino e Divisão dos Dados 65
4.3 Paralelismo 65
4.4 Automação de Hiperparâmetros 68
4.5 Organização dos Experimentos 69
4.6 Desvios em Relação ao Projeto 69

5 RESULTADOS 70
5.1 Protocolo de Execução 70
5.2 Linha de Base 70
5.3 Organização do Código 71

5.4 Segmentadores 71
5.5 Automação de Hiperparâmetros 73
5.6 Paralelismo 75
5.7 Verificação dos Requisitos 76
5.8 Discussão dos Resultados 77

6 CONSIDERAÇÕES FINAIS 78
6.1 Objetivos 78
6.2 Hipóteses 79
6.3 Limitações e Trabalhos Futuros 79

REFERÊNCIAS 80

1 INTRODUÇÃO

As Imagens Hiperespectrais (HSIs) constituem uma tecnologia de sensoriamento remoto que captura informações em centenas de bandas espectrais estreitas e contíguas, permitindo a identificação precisa de materiais e compostos presentes em uma cena (Li et al., 2019; Ahmad et al., 2025). Sua classificação apresenta desafios relacionados à alta dimensionalidade dos dados e à redundância espectral entre bandas adjacentes (Qian, 2020; Lu et al., 2020). As CNNs unidimensionais (CNNs 1D), que convoluem apenas o espectro de cada pixel, consomem menos memória e menos tempo por amostra que as bidimensionais e tridimensionais, ao custo de alguma acurácia, e são uma alternativa para cenários com restrições computacionais (Viel et al., 2023).

A fusão de dados espaciais e espectrais melhora a classificação de HSIs (Imani; Ghassemian, 2020; Souza, 2023). Souza (2023) integrou a uma CNN 1D o identificador do superpixel produzido pelo algoritmo SEEDS, concatenado às características extraídas pelas convoluções, e reportou, em Indian Pines, ganhos de 8,33, 8,79 e 9,47 pontos percentuais em OA, AA e Kappa em relação à mesma rede sem a informação espacial. O trabalho, porém, foi entregue como um protótipo: um notebook executado de forma sequencial, com hiperparâmetros ajustados manualmente e um único algoritmo de segmentação testado.

O processamento paralelo é uma estratégia consolidada para acelerar aplicações de aprendizado profundo (Aach, 2025; Nandan, 2024), e as GPUs, com milhares de núcleos de processamento, executam simultaneamente operações independentes, como os elementos de um mapa de características de uma convolução (Cook, 2012; Jordà; Valero-Lara; Peña, 2019). Na arquitetura de Souza (2023) há ainda uma independência de outra natureza: a segmentação espacial e a extração de características espectrais só se encontram no ponto de fusão e poderiam ser executadas concorrentemente.

Este trabalho parte, portanto, da implementação de Souza (2023) e a otimiza em quatro eixos: a organização do código, o paralelismo, a automação do ajuste de hiperparâmetros e a avaliação de outros algoritmos de segmentação. Os três últimos correspondem a trabalhos futuros sugeridos pelo próprio autor (Souza, 2023); o primeiro é pré-requisito para que os ganhos dos demais sejam atribuíveis. Em todos os eixos, a rede é a CNN 1D definida no projeto (Capítulo 3), com quatro convoluções de 16 a 128 filtros, *max-pooling* e camadas densas de 256 e 128 neurônios; a CNN original aparece apenas como comparação na linha de base. Os ganhos de acurácia são medidos contra a configuração original, isto é, os hiperparâmetros de Souza (2023) com a fusão SEEDS, e os de tempo, contra a mesma etapa executada sem a estratégia avaliada.

1.1 PROBLEMATIZAÇÃO

A arquitetura de Souza (2023) apresenta três limitações. A primeira é o processamento sequencial: as etapas de segmentação e classificação são executadas em série mesmo onde não dependem uma da outra. A segunda é o ajuste manual dos hiperparâmetros da rede e do segmentador: o próprio autor relata ter alterado, por conjunto de dados, a paciência da parada antecipada e outros parâmetros de treino (Souza, 2023), e cada nova cena exige repetir esse ajuste à mão. A terceira é que o algoritmo de segmentação foi escolhido sem comparação com alternativas.

Há um problema anterior a esses três: para que qualquer otimização seja atribuível, é preciso reproduzir o trabalho original de forma auditável. Um notebook com estado global e sem semente fixada para o treino da rede não permite afirmar que uma diferença medida vem da otimização, e não de uma execução diferente (seção 5.2).

1.1.1 Solução Proposta

A solução proposta consiste em reimplementar o trabalho de Souza (2023) em um pacote Python modular e determinístico, que preserve o PCA, a divisão dos dados e os hiperparâmetros de treino do original e use a CNN 1D definida no projeto, e, sobre essa base, implementar e medir: (*i*) estratégias de paralelismo por partição da cena e na fusão concorrente, que sobrepõe a segmentação à extração das características espectrais; (*ii*) a busca automática de hiperparâmetros da CNN e do segmentador, por busca em grade e por Otimização Bayesiana com processo gaussiano; e (*iii*) a comparação de três algoritmos de segmentação (SEEDS, LSC e Watershed), com o LSC e o Watershed calibrados para gerar o mesmo número de regiões que o SEEDS, com a configuração original e sob otimização conjunta com a CNN.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Analisar o impacto de técnicas de segmentação, parametrização automática e processamento paralelo na classificação de imagens hiperespectrais utilizando redes neurais convolucionais unidimensionais.

1.2.2 Objetivos Específicos

1. Reimplementar o fluxo de classificação de Souza (2023) de forma modular e reprodutível, com a CNN 1D definida no projeto, preservando o PCA, a divisão dos dados e os hiperparâmetros de treino da implementação original e garantindo resultados idênticos entre execuções com a mesma configuração;

2. Explorar algoritmos de segmentação espacial (SEEDS, LSC e Watershed) para o processamento de HSIs e avaliar seu impacto na classificação;
3. Implementar métodos de ajuste automático de hiperparâmetros (busca em grade e Otimização Bayesiana com processo gaussiano) para a arquitetura de classificação;
4. Implementar estratégias de processamento paralelo para acelerar as etapas de segmentação e classificação espectral, explorando a independência de dados entre elas e a execução concorrente via threads e processos;
5. Disponibilizar uma análise comparativa do fluxo de classificação de HSIs baseando-se em métricas de acurácia (OA, AA, Kappa) e custo computacional.

1.3 METODOLOGIA

Quanto aos objetivos, a pesquisa é exploratória e descritiva (Gil, 2002). A abordagem é quantitativa, e as conclusões foram obtidas de maneira indutiva, a partir da comparação das métricas de desempenho entre configurações.

Foi realizada uma pesquisa bibliográfica sobre aprendizado profundo (LeCun; Bengio; Hinton, 2015; Goodfellow; Bengio; Courville, 2016), processamento de imagens hiperespectrais (Li et al., 2019; Ahmad et al., 2025; Qian, 2020), redes neurais convolucionais (Lecun et al., 1998; Krizhevsky; Sutskever; Hinton, 2012), otimização de hiperparâmetros (Wu et al., 2019; Liashchynskyi; Liashchynskyi, 2019; Akiba et al., 2019), segmentação de imagens (Van den Bergh et al., 2013; Li; Chen, 2015; Najman; Schmitt, 1996), concorrência (Silberschatz; Galvin; Gagne, 2018; Amdahl, 1967) e paralelização computacional (Aach, 2025; Nandan, 2024; Cook, 2012). Em seguida, o trabalho original foi reimplementado em PyTorch, com a CNN 1D do projeto, e as técnicas escolhidas foram implementadas sobre essa base, com as variáveis de execução reunidas em um único arquivo de configuração. A avaliação foi feita por uma campanha experimental com três conjuntos de dados de referência (Indian Pines, University of Pavia e Salinas Valley), organizada em oito experimentos, numerados de 01 a 08, cujos resultados ficam registrados em arquivos versionados.

1.4 ESTRUTURA DO TRABALHO

O Capítulo 2 apresenta a fundamentação teórica e os trabalhos relacionados. O Capítulo 3 define o projeto, com os requisitos e o plano de verificação. O Capítulo 4 descreve a implementação e os desvios em relação ao projeto. O Capítulo 5 expõe os resultados, organizados pelos quatro eixos de otimização, e a verificação dos requisitos. Por fim, o Capítulo 6 retoma os objetivos e as hipóteses e aponta trabalhos futuros.

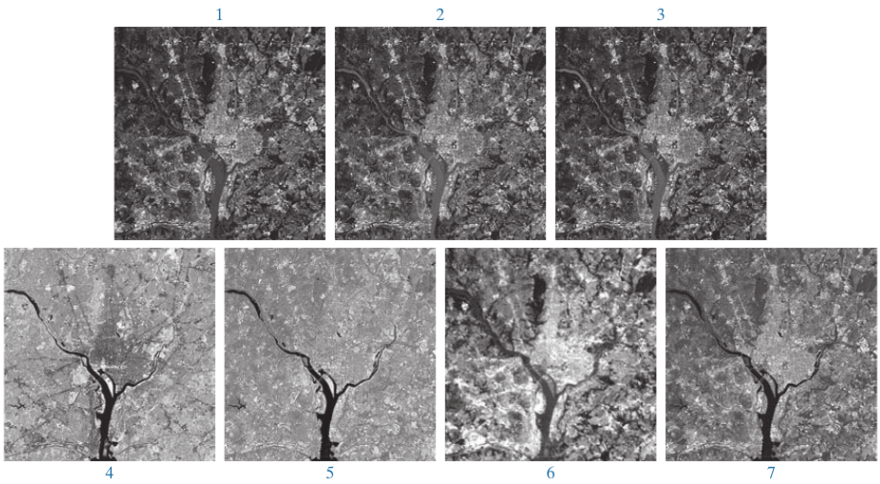
2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos usados no restante do trabalho. Começa pelas imagens hiperespectrais e pela redução de dimensionalidade, passa pelo processamento de imagens, pela segmentação por superpixels e pelas métricas de avaliação, e chega ao aprendizado de máquina e às redes neurais convolucionais unidimensionais usadas na classificação espectral. Em seguida trata do processamento paralelo em GPUs, da concorrência em CPU e da otimização de hiperparâmetros, e termina com os trabalhos relacionados que servem de ponto de partida para a proposta.

2.1 IMAGEM HIPERESPECTRAL

A imagem hiperespectral (HSI), dado de entrada da classificação estudada neste trabalho, combina espectroscopia e imagem e registra informação tanto no domínio espacial quanto no espectral (Li et al., 2019). Imagens multiespectrais capturam poucas bandas largas do espectro; as HSIs registram centenas de bandas estreitas e contíguas, tipicamente na faixa de 400 a 2500 nanômetros (Qian, 2020), o que permite analisar em detalhe a composição dos materiais de uma cena. O espectro eletromagnético pode ser organizado pela energia por fóton, dos raios gama (maior energia) às ondas de rádio (menor energia) (Gonzalez; Woods, 2018). A Figura 1 mostra imagens multiespectrais do satélite LANDSAT da área de Washington D.C., uma por banda temática. Cada banda serve a um uso: a do azul visível é a que mais penetra na água, e a do infravermelho próximo é usada no mapeamento de biomassa e de linhas de costa (Gonzalez; Woods, 2018). A HSI leva essa separação a centenas de bandas e tem aplicações em sensoriamento remoto, agricultura de precisão e diagnóstico médico (Lu et al., 2020).

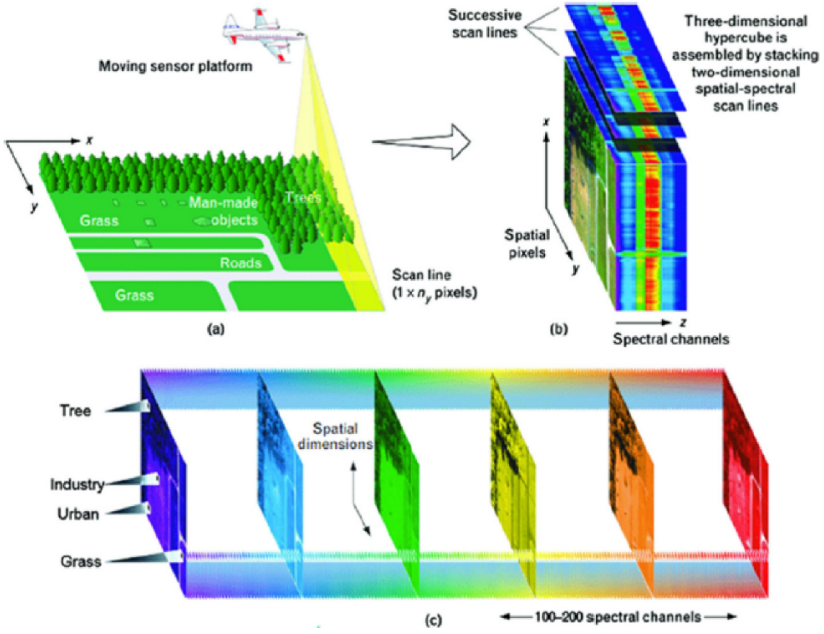
Figura 1 – Imagens de satélite LANDSAT da área de Washington D.C. em diferentes bandas espectrais temáticas



Fonte: Gonzalez; Woods, 2018

Uma HSI tem alta resolução espectral, mas baixa resolução espacial (Viel, 2019), já que o sensor concentra a captura de cada ponto da cena em centenas de bandas estreitas e contíguas. A Figura 2 mostra como o dado se forma: a plataforma do sensor, em movimento, captura linhas de varredura sucessivas, que são empilhadas em um hipercubo com duas dimensões espaciais e uma espectral.

Figura 2 – Estrutura de dados de uma imagem hiperespectral: (a) aquisição por sensor em movimento; (b) formação do hipercubo tridimensional; (c) dimensões espaciais e canais espectrais



Fonte: Qian, 2020

A alta dimensionalidade das HSIs traz três desafios principais à sua análise: (1) a alta redundância de dados, causada pela forte correlação entre bandas espectrais adjacentes; (2) a variabilidade das assinaturas hiperespectrais, que podem mudar com as condições atmosféricas ou de iluminação; e (3) a maldição da dimensionalidade (*curse of dimensionality*), em que o grande número de bandas prejudica o desempenho dos algoritmos de classificação e exige um número exponencialmente maior de amostras de treinamento (Ahmad et al., 2025). As etapas básicas da análise de HSIs incluem pré-processamento, extração e seleção de características, e classificação ou desmistura (*unmixing*) espectral.

2.1.1 Redução de Dimensionalidade

O número de bandas das HSIs e a correlação entre bandas adjacentes motivam a redução de dimensionalidade antes da classificação. A Análise de Componentes Principais (PCA) projeta os dados sobre as direções de maior variância e, com poucas componentes, preserva a maior parte da variância dos dados originais (Jolliffe, 2002). A projeção é definida pela equação:

$$z_i^t = w_i^T x^t, \quad i = 1, \ldots, k, \quad t = 1, \ldots, N \tag{1}$$

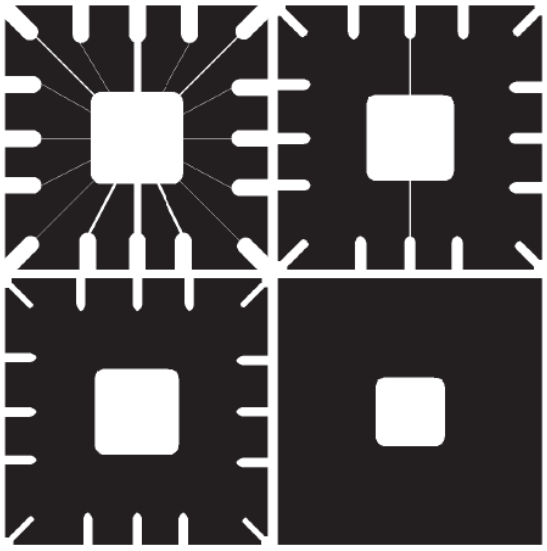
onde x^t são os dados de entrada do pixel t, w_i são os autovetores correspondentes aos maiores autovalores, z_i^t são as componentes principais projetadas, k é o número de componentes principais retidas, e N é o número total de pixels. O algoritmo envolve: (1) centralização dos dados, subtraindo a média espectral; (2) cálculo da matriz de covariância; (3) decomposição da matriz de covariância em autovetores e autovalores; e (4) projeção dos dados originais nos autovetores correspondentes aos maiores autovalores (Jolliffe, 2002). Neste trabalho, o número k de componentes retidas é um dos hiperparâmetros do espaço de busca (Quadro 10).

2.2 PROCESSAMENTO DE IMAGENS

Uma imagem pode ser definida como uma função bidimensional, $f(x, y)$, onde x e y são coordenadas espaciais, e a amplitude de f em qualquer par de coordenadas (x, y) é chamada de intensidade ou nível de cinza da imagem naquele ponto (Gonzalez; Woods, 2018). Quando x, y e os valores de intensidade de f são quantidades finitas e discretas, a imagem é chamada de imagem digital. O campo do processamento digital de imagens refere-se ao processamento de imagens digitais por meio de um computador digital.

O processamento de imagens pode ser categorizado em três níveis: baixo nível (pré-processamento, redução de ruído, realce de contraste), nível médio (segmentação, descrição de objetos, classificação) e alto nível (análise de cena e reconhecimento) (Gonzalez; Woods, 2018). A Figura 3 exemplifica uma operação morfológica de erosão, um tipo de processamento de baixo nível comumente utilizado para remover componentes indesejados de imagens binárias.

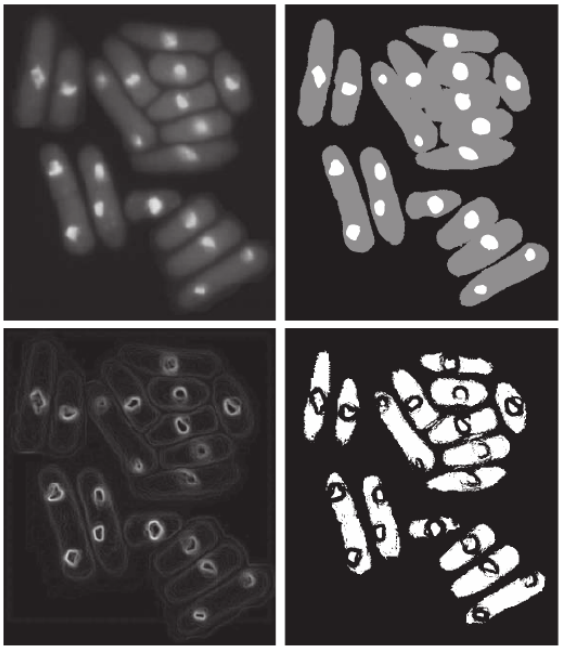
Figura 3 – Exemplo de operação de erosão morfológica aplicada a uma imagem binária usando elementos estruturantes de diferentes tamanhos



Fonte: Gonzalez; Woods, 2018

A Figura 4 exemplifica o processamento de nível médio, mostrando a segmentação e identificação de células individuais em imagens microscópicas.

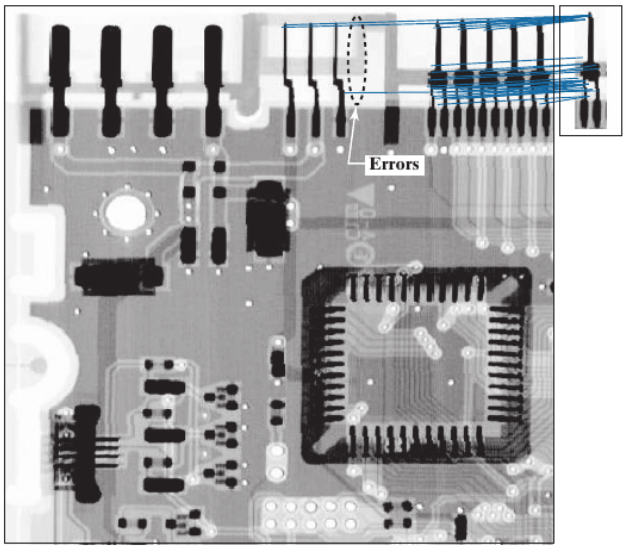
Figura 4 – Exemplo de processamento de nível médio: segmentação de células em imagens microscópicas



Fonte: Gonzalez; Woods, 2018

A Figura 5 exemplifica o processamento de alto nível, mostrando o reconhecimento e correspondência de características entre imagens usando o algoritmo SIFT.

Figura 5 – Exemplo de processamento de alto nível: correspondência de características usando SIFT, mostrando 41 pares encontrados com apenas 3 erros reais

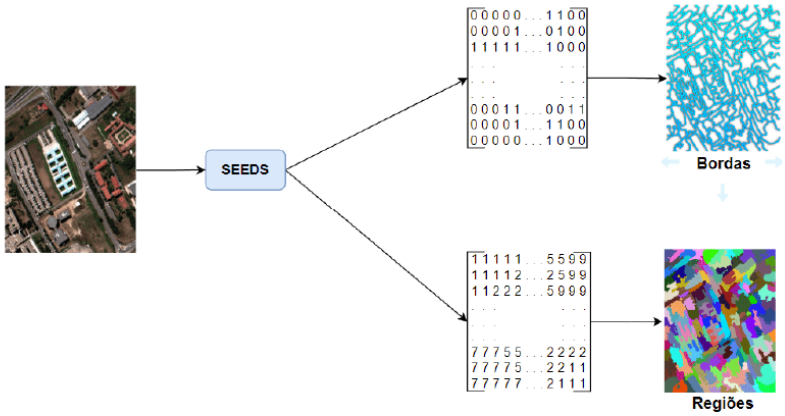


Fonte: Gonzalez; Woods, 2018

2.2.1 Segmentação de Imagens

No fluxo de classificação deste trabalho, a segmentação extrai a informação espacial que complementa as características espectrais. A segmentação de imagens é o processo de particionar uma imagem em múltiplos segmentos ou regiões de interesse (Feng et al., 2019), e sua função é simplificar a representação da imagem em algo mais fácil de analisar. No domínio das HSIs, a segmentação usa a informação espacial para auxiliar a classificação, o que é útil quando objetos distintos apresentam assinaturas espectrais parecidas, mas estruturas geométricas ou texturas diferentes (Souza, 2023). Em Indian Pines, por exemplo, milho e soja aparecem cada um em três classes, conforme o manejo do solo; os espectros dessas classes diferem pouco entre si, mas cada uma ocupa talhões contíguos, e a região a que o pixel pertence pode ajudar a separar o que o espectro sozinho confunde. A Figura 6 ilustra os dois principais tipos de mapa de segmentação gerados pelo algoritmo SEEDS: o mapa de bordas, que identifica os contornos entre superpixels, e o mapa de regiões, que atribui rótulos únicos a cada segmento homogêneo da imagem.

Figura 6 – Tipos de mapa de segmentação: bordas (superior) e regiões (inferior) gerados pelo algoritmo SEEDS



Fonte: Souza, 2023

Um método de segmentação morfológica utiliza os resíduos de transformadas de abertura e fechamento morfológicas baseadas em uma métrica geodésica (Pesaresi; Benediktsson, 2001). Essa abordagem introduz o conceito de nivelamento (*leveling*). A reconstrução em tons de cinza de uma imagem g a partir de um marcador f é obtida pelo uso iterativo de dilatações geodésicas elementares até a idempotência:

$$R_g^{\delta}(f) = \delta_g^{(i)}(f) \tag{2}$$

onde $R_g^{\delta}(f)$ é a reconstrução geodésica do marcador f sob a imagem de referência g, e $\delta_g^{(i)}(f)$ representa a dilatação geodésica elementar aplicada iterativamente até atingir idempotência (estabilidade). A característica morfológica multiescala de um ponto na imagem é definida pelo tamanho do elemento estruturante que possui o maior valor associado na derivada do perfil morfológico. O vetor da derivada do perfil morfológico (DMP) é:

$$\Delta(x) = (\Delta_0(x), ..., \Delta_\lambda(x), ..., \Delta_N(x)) \tag{3}$$

onde $\Delta(x)$ é o vetor de derivadas do perfil morfológico no ponto x, $\Delta_\lambda(x)$ representa a derivada para o elemento estruturante de tamanho λ, e N é o número total de escalas.

O SEEDS (*Superpixels Extracted via Energy-Driven Sampling*) parte de uma partição da imagem em grade regular e refina as fronteiras dos superpixels por *hill-climbing*: blocos de

pixels, e depois pixels isolados, passam para um superpixel vizinho sempre que a troca aumenta uma função de energia (Van den Bergh et al., 2013):

$$E(s) = H(s) + \gamma G(s) \tag{4}$$

onde $E(s)$ é a energia da segmentação s, $H(s)$ é o termo de histograma, que mede a homogeneidade de cor dentro de cada superpixel, $G(s)$ é o termo de contorno, que favorece fronteiras suaves, e γ controla o equilíbrio entre cor e contornos.

Além do SEEDS, dois outros algoritmos de superpixels são usados neste trabalho. O LSC (*Linear Spectral Clustering*) mapeia cada pixel, descrito pela cor e pela posição, para um espaço de características de dimensão mais alta, no qual um *k-means* ponderado aproxima o critério de cortes normalizados em grafos, o que tende a produzir bordas mais aderentes aos contornos da imagem; o tamanho médio das regiões e o peso da compacidade são seus parâmetros principais (Li; Chen, 2015). O Watershed trata a magnitude do gradiente como um relevo topográfico e o inunda a partir de marcadores; as fronteiras entre regiões surgem onde águas de bacias diferentes se encontram, e o número de regiões acompanha o número de marcadores (Najman; Schmitt, 1996). Todos eles, assim como o SEEDS, recebem como entrada uma imagem de três canais: no caso das HSIs, uma composição das três primeiras componentes principais. Como a informação que o identificador do superpixel leva à rede depende de quantas regiões a cena tem, o LSC e o Watershed foram configurados para gerar, em cada cena, o mesmo número de regiões que o SEEDS com os parâmetros do trabalho original; a comparação entre os três mede, assim, a forma das regiões, e não a sua quantidade (subseção 4.2.1).

2.2.2 Métricas de Avaliação

As métricas de classificação de HSIs derivam da matriz de confusão, que cruza, para cada pixel avaliado, a classe verdadeira com a classe prevista (Congalton, 1991). Sejam n_{ij} o número de pixels da classe verdadeira i atribuídos à classe j, c o número de classes, N o total de pixels avaliados, e $n_{i+} = \sum_j n_{ij}$ e $n_{+i} = \sum_j n_{ji}$ os totais da linha e da coluna da classe i. As três métricas usadas neste trabalho são as seguintes.

Acurácia Geral (Overall Accuracy - OA):

$$OA = \frac{1}{N} \sum_{i=1}^{c} n_{ii} \tag{5}$$

isto é, a proporção de pixels classificados corretamente (Congalton, 1991). Com duas classes, a expressão se reduz a $(vp+vn)/(vp+vn+fp+fn)$, em que vp, vn, fp e fn são os verdadeiros positivos, os verdadeiros negativos, os falsos positivos e os falsos negativos.

Acurácia Média (Average Accuracy - AA):

$$AA = \frac{1}{c}\sum_{i=1}^{c}\frac{n_{ii}}{n_{i+}} \tag{6}$$

onde n_{ii}/n_{i+} é a acurácia da classe i, isto é, a fração dos seus pixels classificada corretamente. Como cada classe pesa o mesmo, a AA pune o classificador que acerta só as classes grandes. Por exemplo, um classificador que acerta 85 dos 90 pixels de uma classe frequente, mas só 4 dos 10 pixels de uma classe rara e 8 dos 10 de outra, tem OA de 0,88 e AA de 0,71. Em Indian Pines, cujas classes vão de 20 a 2.455 pixels rotulados, a distância entre as duas métricas é grande.

Coeficiente Kappa (κ):

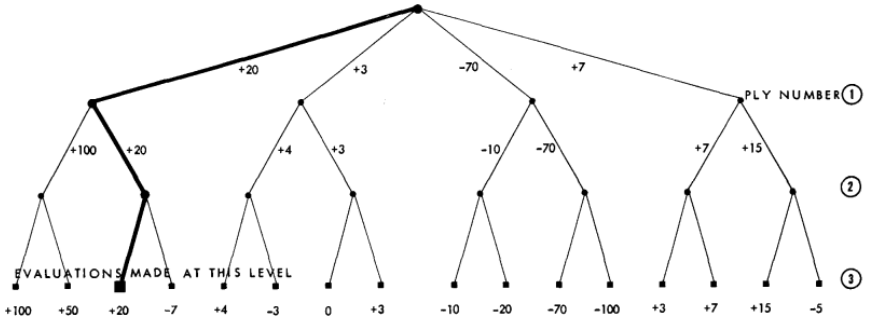
$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{7}$$

onde p_o é a concordância observada, igual à OA, e $p_e = \frac{1}{N^2}\sum_{i=1}^{c} n_{i+}\, n_{+i}$ é a concordância esperada por acaso, calculada pelos totais de linha e de coluna. Proposto por Cohen (1960), o coeficiente mede a concordância entre a classificação e o Ground Truth (GT) descontando a parcela que ocorreria por acaso: vale 1 para concordância perfeita e 0 quando a concordância é a esperada por acaso (Congalton, 1991).

2.3 APRENDIZADO DE MÁQUINA

O Aprendizado de Máquina, do inglês *Machine Learning* (ML), é o campo de estudo que dá aos computadores a capacidade de aprender sem serem explicitamente programados (Samuel, 1959). O aprendizado profundo, ao qual pertencem as CNNs usadas neste trabalho, é um de seus ramos (LeCun; Bengio; Hinton, 2015). Um programa de computador aprende a partir de uma experiência (E) com respeito a uma classe de tarefas (T) e uma medida de desempenho (P) se o seu desempenho nas tarefas em T, medido por P, melhora com a experiência E (Mitchell, 2017). A Figura 7 mostra um exemplo clássico, o programa de damas de Samuel (1959), que avalia os movimentos possíveis por meio de uma árvore de jogadas e escolhe o ramo de maior pontuação.

Figura 7 – Diagrama simplificado mostrando como as avaliações são respaldadas através da árvore de movimentos possíveis para chegar ao melhor próximo movimento



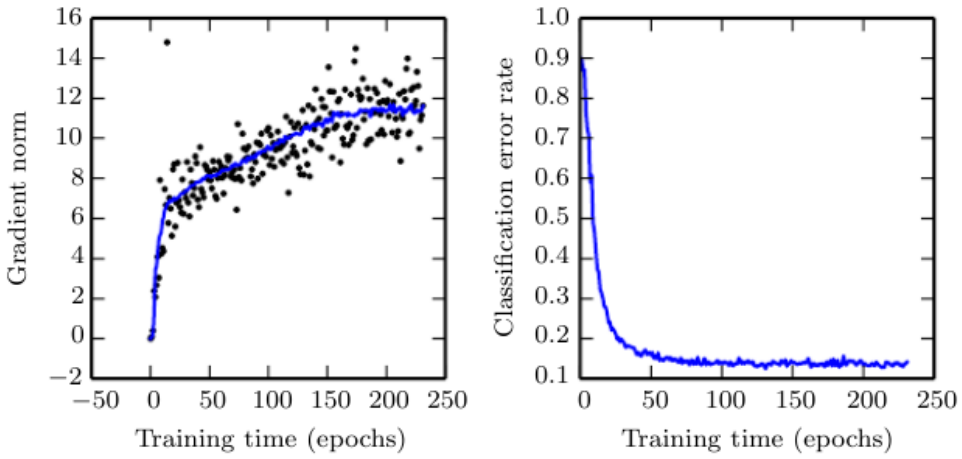
Fonte: Samuel, 1959

2.3.1 Desafios Centrais: Generalização e Sobreajuste

O desafio principal é garantir que o desempenho de um modelo (P) melhore não apenas nos dados de treinamento (E), mas também em dados novos. Quando isso não ocorre, há sobreajuste, do inglês *overfitting* (OF): o modelo se ajusta demais aos dados de treinamento e captura ruído em vez do padrão subjacente (Ding; Xiang, 2003), com bom desempenho no treinamento e desempenho ruim em entradas novas. Por isso o desempenho é acompanhado em um conjunto de validação, separado do treino. A Figura 8 traz um exemplo de Goodfellow, Bengio e Courville (2016): durante o treino de uma rede convolucional, a norma do gradiente cresce, sinal de que o otimizador não chega a um ponto crítico, e ainda assim o erro de classificação na validação cai a um nível baixo. O que indica se o modelo generaliza é esse erro, medido fora do treino, e não a convergência do otimizador.

As estratégias de regularização restringem a complexidade efetiva do modelo para reduzir o OF (Goodfellow; Bengio; Courville, 2016). Três delas aparecem neste trabalho: o *dropout*, que desativa ao acaso uma fração dos neurônios a cada passo de treino e impede que eles se coadaptem (Srivastava et al., 2014), usado nas duas camadas densas ocultas da rede; o *weight decay*, que penaliza a norma dos pesos; e a parada antecipada, que interrompe o treino quando o desempenho na validação deixa de melhorar (Goodfellow; Bengio; Courville, 2016).

Figura 8 – Comportamento da descida de gradiente durante o treinamento de uma rede convolucional: (Esquerda) distribuição das normas do gradiente ao longo do tempo; (Direita) taxa de erro de classificação no conjunto de validação

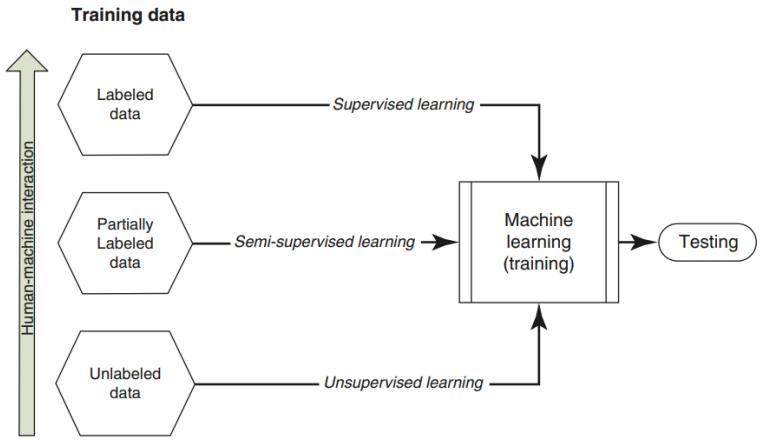


Fonte: Goodfellow; Bengio; Courville, 2016

2.3.2 Paradigmas de Aprendizagem

A natureza dos dados e do problema determina qual paradigma de aprendizado usar (Naqa; Murphy, 2015). A Figura 9 ilustra os três principais paradigmas e a relação de cada um com o tipo de dado de treinamento.

Figura 9 – Paradigmas de aprendizagem e sua relação com os tipos de dados de treinamento



Fonte: Naqa; Murphy, 2015

- **Aprendizado Supervisionado**: Estima um mapeamento a partir de amostras de entrada e saída, em que as saídas são rótulos ou valores conhecidos. As tarefas mais comuns são classificação e regressão. É o paradigma deste trabalho: cada pixel rotulado do GT fornece um espectro de entrada e a classe esperada.

- **Aprendizado Não Supervisionado**: O sistema recebe apenas amostras de entrada, sem rótulos, e procura estrutura nos próprios dados, como grupos de amostras semelhantes. Os três algoritmos de superpixels deste trabalho (subseção 2.2.1) são exemplos: particionam a imagem sem usar nenhum rótulo, e o LSC, em particular, agrupa os pixels por um *k-means* ponderado.
- **Aprendizado Semissupervisionado**: Do inglês *Semi-supervised learning* (SSL), situa-se entre os dois paradigmas anteriores e usa dados em que apenas parte das amostras é rotulada.

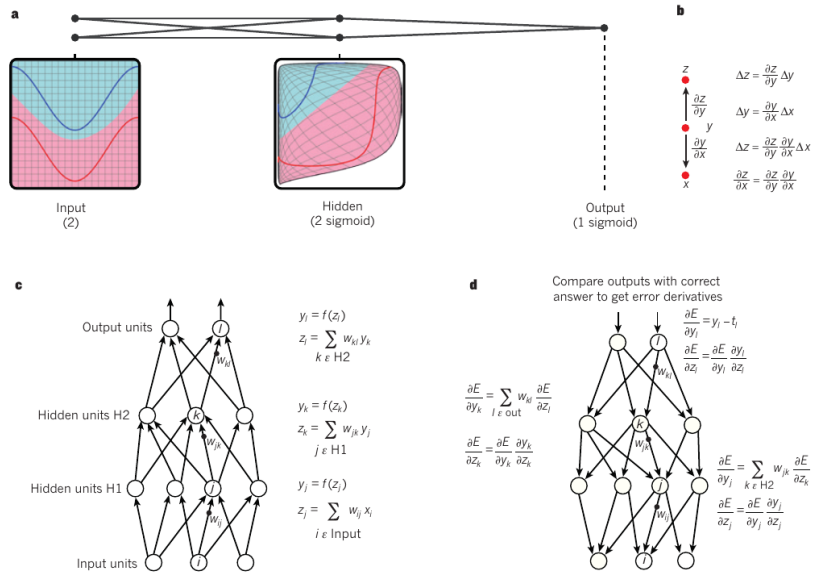
2.3.3 Aprendizado Profundo

Os sistemas tradicionais de reconhecimento de padrões eram formados por dois módulos: um extrator de características projetado manualmente e um classificador treinável, e o desempenho dependia da engenharia dessas características (LeCun; Bengio; Hinton, 2015). O Aprendizado Profundo, do inglês *Deep Learning* (DL), remove essa dependência: modelos compostos por múltiplas camadas de processamento aprendem, diretamente dos dados brutos, representações com múltiplos níveis de abstração (Noor; Ige, 2025). O DL é um método de aprendizado de representação obtido pela composição de módulos simples e não lineares, em que cada camada transforma a representação da camada anterior em uma de nível de abstração superior (LeCun; Bengio; Hinton, 2015). A Figura 11 ilustra a arquitetura de uma rede neural convolucional, com convoluções seguidas da função de ativação ReLU e camadas de *max pooling* que reduzem progressivamente a dimensão espacial. As funções de ativação, como a Unidade Linear Retificada (ReLU), introduzem não linearidades na rede. A ReLU é definida como:

$$f(z) = \max(z, 0) = \begin{cases} z, & \text{se } z > 0 \\ 0, & \text{caso contrário} \end{cases} \tag{8}$$

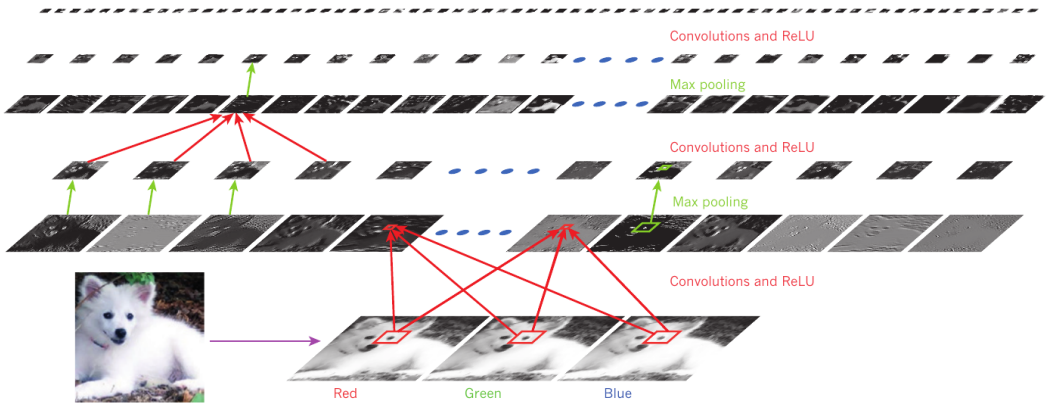
onde z é a entrada da função de ativação. Por não saturar para entradas positivas, a ReLU acelera o treino em relação a funções saturantes, como a sigmoide e a tangente hiperbólica: em uma rede convolucional de quatro camadas treinada no CIFAR-10, a versão com ReLU atingiu 25% de erro de treino seis vezes mais rápido que a versão com tangente hiperbólica (Krizhevsky; Sutskever; Hinton, 2012). O treinamento dessas redes usa o algoritmo de retropropagação, do inglês *backpropagation*, que calcula o gradiente da função de perda em relação a cada peso da rede e permite ajustar os parâmetros por descida de gradiente (Lecun et al., 1998). A Figura 10 mostra como uma rede neural multicamadas pode distorcer o espaço de entrada para tornar as classes linearmente separáveis, e como a retropropagação aplica a regra da cadeia, tanto na propagação para frente (*forward pass*) quanto na retropropagação dos gradientes (*backward pass*).

Figura 10 – Redes neurais multicamadas e backpropagation: (a) transformação do espaço de entrada; (b) regra da cadeia; (c) equações do forward pass; (d) equações do backward pass



Fonte: LeCun; Bengio; Hinton, 2015

Figura 11 – Arquitetura de uma rede neural convolucional mostrando camadas de convolução com ReLU e max pooling



Fonte: LeCun; Bengio; Hinton, 2015

O treinamento de redes neurais profundas usa algoritmos de otimização estocástica para ajustar os parâmetros do modelo; o Adam, usado neste trabalho, é um deles (Kingma; Ba, 2017). O gradiente é definido como:

$$\nabla_\theta L = \frac{\partial L}{\partial \theta} \tag{9}$$

onde ∇_θ é o operador gradiente, L é a função de perda, e θ são os parâmetros do modelo. A função de perda depende da tarefa: em regressão, a escolha usual é o erro quadrático médio; em classificação, como neste trabalho (subseção 4.2.3), usa-se a entropia cruzada (Goodfellow; Bengio; Courville, 2016):

$$L = -\frac{1}{N}\sum_{t=1}^{N}\sum_{i=1}^{c} r_i^t \log y_i^t \tag{10}$$

onde N é o número de amostras, c é o número de classes, r_i^t vale 1 se a amostra t pertence à classe i e 0 caso contrário, e y_i^t é a probabilidade da classe i prevista pelo modelo para a entrada x^t, obtida pela função *softmax* aplicada à saída da última camada.

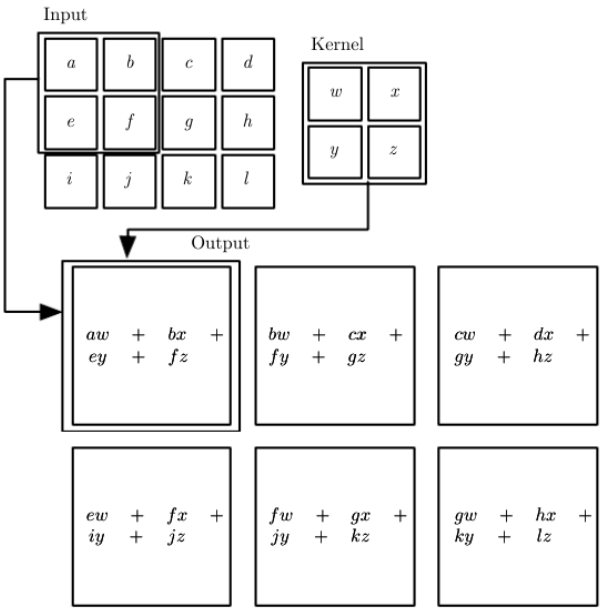
2.3.3.1 Redes Neurais Convolucionais

As CNNs são um tipo especializado de rede neural projetado para processar dados com topologia de grade (Goodfellow; Bengio; Courville, 2016), como o espectro de um pixel, que é uma grade unidimensional. Sua arquitetura se apoia em quatro ideias: conexões locais, pesos compartilhados, agrupamento (*pooling*) e uso de múltiplas camadas (LeCun; Bengio; Hinton, 2015). A operação de convolução em uma dimensão (1D) é definida como:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a) \cdot w(t-a) \tag{11}$$

onde x é o sinal de entrada, w é o *kernel* (filtro) da convolução e $s(t)$ é o resultado da operação na posição t. Na CNN 1D deste trabalho, o sinal de entrada é o espectro de um pixel após o PCA, e t percorre as componentes espectrais. A Figura 12 mostra, no caso bidimensional, o *kernel* aplicado sobre a matriz de entrada.

Figura 12 – Operação de convolução mostrando a aplicação do kernel sobre a matriz de entrada



Fonte: LeCun; Bengio; Hinton, 2015

Em vez de conectar cada neurônio de uma camada a todos os neurônios da camada anterior, os neurônios em uma camada convolucional são conectados apenas a uma região local (conexões locais) (LeCun; Bengio; Hinton, 2015). A entrada líquida (*net input*) em um neurônio convolucional é:

$$z^{\ell}_{x,y} = b^{\ell} + \sum_{l,k} w^{\ell}_{l,k} \cdot a^{\ell-1}_{x-l,y-k} \tag{12}$$

onde $z^{\ell}_{x,y}$ é a entrada na posição (x, y) da camada ℓ, b^{ℓ} é o viés (*bias*), $w^{\ell}_{l,k}$ são os pesos do *kernel* e $a^{\ell-1}_{x-l,y-k}$ são as ativações da camada anterior. Todas as unidades de um mesmo mapa de características compartilham o mesmo conjunto de pesos, o que reduz o número de parâmetros. Essa abordagem assume que um detector de características útil em uma parte da imagem será útil em outra parte. A camada de *pooling* reduz a dimensão espacial dos mapas de características e torna a representação mais compacta e robusta a pequenas translações. Uma CNN típica consiste em estágios de camadas convolucionais seguidas por camadas de *pooling*, e termina em camadas totalmente conectadas para a classificação. Na CNN deste trabalho (Figura 24), um *max-pooling* de tamanho 2 segue cada uma das três primeiras convoluções, que preservam o comprimento do espectro.

Na convolução, cada elemento do mapa de características de saída depende apenas de

uma região local da entrada e dos pesos do *kernel* compartilhado, e não dos demais elementos da mesma camada. Essa independência de dados entre elementos de saída é o que permite calcular a camada em paralelo na GPU (seção 2.4).

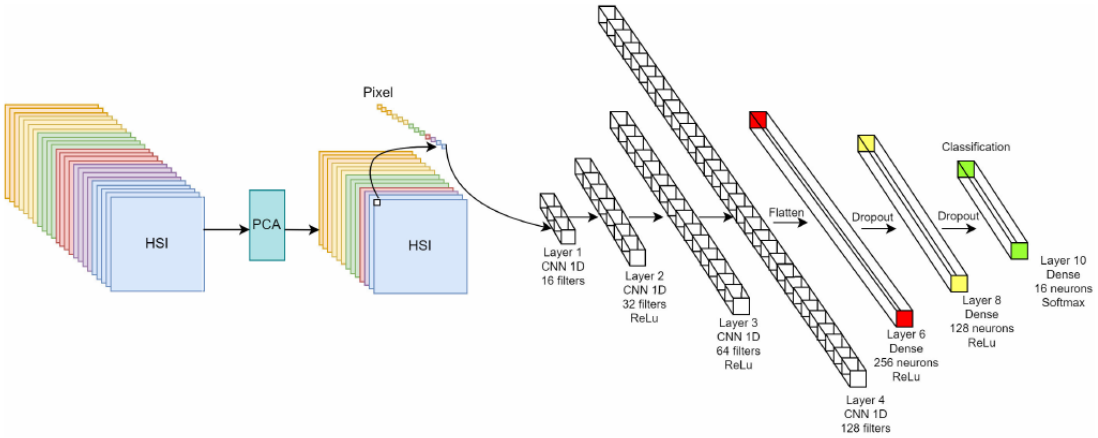
A evolução das arquiteturas de CNNs é marcada pelo aumento progressivo da profundidade. Redes pioneiras como a LeNet-5 demonstraram a eficácia da combinação de camadas convolucionais e de *pooling* no reconhecimento de caracteres (Lecun et al., 1998). A VGG mostrou que redes profundas podiam ser construídas com filtros pequenos (3×3) (Simonyan; Zisserman, 2015). A GoogLeNet introduziu os módulos *Inception*, que executam convoluções de diferentes tamanhos em paralelo (Szegedy et al., 2014).

2.3.3.2 Aplicações de CNNs

As CNNs são usadas em várias aplicações:

- **Classificação de Imagens**: Atribui um único rótulo a uma imagem inteira (Krizhevsky; Sutskever; Hinton, 2012).
- **Segmentação Semântica**: Classifica cada pixel de uma imagem em uma categoria específica (Farabet et al., 2013). Em sensoriamento remoto, é usada no mapeamento de cobertura do solo (Chroni et al., 2024).
- **Detecção de Objetos**: Localiza múltiplos objetos em uma imagem por meio de caixas delimitadoras (*bounding boxes*) e classifica cada um (Sermanet et al., 2014).
- **Análise de Imagens Hiperespectrais**: Em imagens hiperespectrais, que contêm centenas de bandas espectrais, as CNNs extraem características espectrais e espaciais (Li et al., 2019). A Figura 13 ilustra uma arquitetura CNN 1D para classificação de HSIs, do pré-processamento por PCA às camadas convolucionais 1D e densas que produzem a classificação.

Figura 13 – Arquitetura CNN 1D para classificação de imagens hiperespectrais: pré-processamento PCA seguido de 4 camadas convolucionais 1D e camadas densas para classificação



Fonte: Viel et al., 2023

2.4 PROCESSAMENTO PARALELO EM GPUS

Neste trabalho, a CNN é treinada em GPU, exceto nas medições de paralelismo, feitas em CPU (seção 5.6), e a segmentação é sempre executada em CPU. A arquitetura da GPU e o modelo de programação CUDA explicam por que as convoluções da rede se paralelizam bem, e a independência entre as etapas do fluxo define as demais estratégias de paralelização.

2.4.1 Arquitetura de GPUs e CUDA Cores

As Unidades de Processamento Gráfico (GPUs) evoluíram de aceleradores gráficos especializados para processadores paralelos de propósito geral (Cook, 2012). Enquanto uma CPU tem tipicamente dezenas de núcleos otimizados para execução sequencial com baixa latência, uma GPU NVIDIA tem milhares de núcleos de processamento mais simples, os *CUDA cores*, organizados em unidades chamadas *Streaming Multiprocessors* (SMs) (Cook, 2012).

Cada *CUDA core* executa operações aritméticas de ponto flutuante, como soma, multiplicação e a operação fusionada de multiplicação e soma (FMA). Os *CUDA cores* de um SM seguem o modelo SIMT (*Single Instruction, Multiple Threads*): grupos de 32 threads, chamados *warps*, executam a mesma instrução ao mesmo tempo sobre dados diferentes (Cook, 2012). Essa arquitetura é eficiente para operações com paralelismo de dados, em que a mesma operação é aplicada a muitos elementos independentes.

A plataforma CUDA (*Compute Unified Device Architecture*), desenvolvida pela NVIDIA, fornece um modelo de programação que permite explorar o paralelismo das GPUs em aplicações de computação geral (Cook, 2012). O modelo de execução organiza as threads em três níveis:

threads individuais são agrupadas em blocos, e os blocos, em grades. Como o hardware distribui os blocos entre os SMs disponíveis, o mesmo programa se adapta a GPUs com números diferentes de SMs (Nickolls et al., 2008).

2.4.2 Paralelização de Operações de Convolução

A operação de convolução em CNNs apresenta características intrínsecas que a tornam altamente paralelizável (Jordà; Valero-Lara; Peña, 2022). Para uma camada convolucional com entrada X de dimensões (N, C_{in}, H, W) (lote, canais de entrada, altura, largura), filtros K de dimensões $(C_{out}, C_{in}, k_h, k_w)$ e saída Y de dimensões $(N, C_{out}, H_{out}, W_{out})$, cada elemento de saída é calculado como:

$$Y[n, c_{out}, i, j] = \sum_{c_{in}=0}^{C_{in}-1} \sum_{m=0}^{k_h-1} \sum_{l=0}^{k_w-1} X[n, c_{in}, i+m, j+l] \cdot K[c_{out}, c_{in}, m, l] + b[c_{out}] \qquad (13)$$

Nenhum elemento $Y[n, c_{out}, i, j]$ depende do cálculo de outro elemento da saída; os $N \times C_{out} \times H_{out} \times W_{out}$ elementos podem, portanto, ser calculados simultaneamente, se houver recursos computacionais suficientes. Na CNN 1D, a dimensão da altura desaparece e a propriedade se mantém: cada posição do mapa de características ao longo do espectro pode ser calculada em paralelo.

Na prática, *frameworks* de aprendizado profundo como TensorFlow e PyTorch usam a biblioteca cuDNN da NVIDIA, que escolhe, para cada configuração da camada, um entre vários algoritmos de convolução (Jordà; Valero-Lara; Peña, 2019).

2.4.3 Estratégias de Paralelização para o Fluxo de Classificação

O treinamento e a inferência de CNNs admitem paralelismo em mais de um nível (Aach, 2025; Nandan, 2024). Os três a seguir são os relevantes para o fluxo de classificação de HSIs.

Paralelismo de Dados: Réplicas do modelo processam subconjuntos diferentes dos dados ao mesmo tempo, e os gradientes são agregados para atualizar os pesos. A MirroredStrategy do TensorFlow e a DataParallel do PyTorch implementam essa estratégia, mais útil quando o conjunto de dados é grande e há várias GPUs disponíveis (Nandan, 2024). A partição da cena, uma das estratégias deste trabalho, também divide os dados entre unidades de execução, mas sem agregar gradientes: cada parte da cena é processada por uma thread ou por um processo (seção 4.3).

Paralelismo Intra-Camada: Dentro de uma camada convolucional, os elementos de

saída são calculados em paralelo nos *CUDA cores*. As implementações em GPU, como a cuDNN, exploram esse paralelismo automaticamente, sem intervenção do programador.

Paralelismo entre Etapas: Etapas que não dependem uma da outra podem ser executadas concorrentemente. No fluxo de Souza (2023), isso ocorre em dois pontos. Na preparação dos dados, o ramo espectral, que reduz o espectro por PCA, e o ramo espacial, que compõe as três primeiras componentes principais e segmenta a imagem, partem da mesma cena e só se reúnem na montagem das entradas da rede. Na inferência, a segmentação, executada em CPU pelo OpenCV, e as convoluções da rede também são independentes: as convoluções não usam o mapa de segmentação, e a sincronização só é necessária na concatenação, que precisa do identificador do superpixel de cada pixel. Essas duas oportunidades correspondem, neste trabalho, ao paralelismo entre os ramos do pré-processamento e à fusão concorrente (seção 4.3), ambas executadas com threads da biblioteca padrão do Python (Gorelick; Ozsvald, 2020): o PCA, o OpenCV e as convoluções do PyTorch liberam o GIL no trabalho pesado, e as threads compartilham os dados sem cópia (seção 2.5).

2.5 CONCORRÊNCIA EM CPU

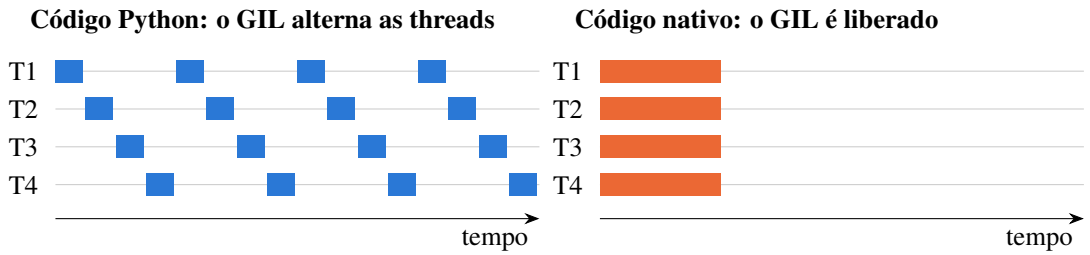
O paralelismo entre etapas exige os conceitos clássicos de concorrência dos sistemas operacionais. Na fusão espectral-espacial, a segmentação, executada em CPU pelo OpenCV, e as convoluções da rede só se encontram no ponto de fusão.

2.5.1 Threads, Processos e o GIL

Um processo é um programa em execução com espaço de endereçamento próprio; uma thread é um fluxo de execução dentro de um processo, e as threads de um mesmo processo compartilham sua memória (Silberschatz; Galvin; Gagne, 2018). Criar uma thread é barato e não copia dados; processos são isolados, e trocar dados entre eles exige serializá-los e copiá-los, com custo proporcional ao volume transferido.

No CPython, o interpretador de referência do Python, o *Global Interpreter Lock* (GIL) garante que apenas uma thread execute *bytecode* Python por vez (Gross, 2023). Código Python puro em várias threads, portanto, não executa em paralelo, enquanto código nativo que libera o GIL durante o cálculo, como as rotinas do NumPy, o OpenCV e os *kernels* do PyTorch, executa (Figura 14). O ganho das threads depende da fração do trabalho que acontece fora do interpretador.

Figura 14 – Quatro threads de um processo executando código Python e código nativo que libera o GIL



Fonte: Elaborada pelo autor

2.5.2 Exclusão Mútua e Variáveis de Condição

Quando duas threads acessam o mesmo dado e ao menos uma o modifica, o resultado pode depender da ordem de execução, situação chamada de condição de corrida. O trecho que acessa o dado compartilhado é a seção crítica, protegida em geral por um mutex (de *mutual exclusion*): uma trava que só uma thread detém por vez (Silberschatz; Galvin; Gagne, 2018). O mutex garante exclusão, mas não ordem: se a thread consumidora adquirir a trava antes da produtora, lerá o dado ainda não produzido. Para impor ordem, usa-se uma variável de condição, associada ao mutex e a um predicado: a consumidora verifica o predicado e, se ele for falso, espera com a trava liberada, até que a produtora atualize o dado e a notifique. Esse conjunto corresponde ao conceito de monitor (Hoare, 1974). A barreira, outra primitiva usada neste trabalho, bloqueia um grupo de threads até que todas cheguem ao mesmo ponto.

2.5.3 Lei de Amdahl e *Speedup*

O *speedup* é a razão entre o tempo de uma versão de referência e o da versão paralela, e a escolha da referência é parte da medição: comparar uma versão paralela em CPU com uma sequencial em GPU mede também a diferença de dispositivo. Pela lei de Amdahl (Amdahl, 1967), se uma fração p do tempo de execução pode ser dividida entre N unidades, o *speedup* máximo é:

$$S(N) = \frac{1}{(1-p) + \dfrac{p}{N}} \tag{14}$$

Dois corolários orientam este trabalho. Paralelizar uma etapa que responde por 1% do tempo total muda esse tempo em no máximo 1%. E, quando duas etapas independentes de durações T_1 e T_2 passam a ser executadas concorrentemente, o *speedup* não pode exceder:

$$S_{\max} = \frac{T_1 + T_2}{\max(T_1, T_2)} \tag{15}$$

que vale 2 apenas quando as etapas têm a mesma duração. A distância até esse teto vem da sobrecarga (*overhead*) de coordenar a execução, isto é, criar threads ou processos, serializar dados e sincronizar, e da contenção: tarefas que disputam a banda de memória, o *cache* ou o GIL ficam mais lentas do que seriam sozinhas.

2.6 OTIMIZAÇÃO DE HIPERPARÂMETROS

Hiperparâmetros são as escolhas feitas antes do treino, como a taxa de aprendizado, o *dropout*, o tamanho do lote, o número de componentes do PCA e os parâmetros do segmentador; os parâmetros, os pesos, são o que o treino aprende. A Otimização de Hiperparâmetros (HPO) automatiza a escolha: cada avaliação de uma combinação, chamada *trial*, consiste em treinar e validar um modelo (Wu et al., 2019).

A busca em grade avalia todas as combinações de um espaço discreto e fica inviável rapidamente, uma vez que o número de combinações é o produto do número de valores de cada hiperparâmetro (Liashchynskyi; Liashchynskyi, 2019). Com três valores em cada um dos sete eixos deste trabalho (segmentador, configuração do segmentador, taxa de aprendizado, *dropout*, *weight decay*, tamanho do lote e componentes do PCA), seriam 2.187 combinações por cena. Por isso, neste trabalho, a grade cobre só o segmentador, a taxa de aprendizado e o *dropout*, em 27 combinações, com os demais eixos no valor da configuração original (seção 4.4).

A Otimização Bayesiana, aplicada aqui ao espaço inteiro, usa os *trials* já observados para decidir o próximo. Ela modela a função objetivo, a acurácia de validação, por um processo gaussiano: uma distribuição sobre funções definida por uma média e por um núcleo de covariância, que expressa quanto duas combinações próximas tendem a ter valores parecidos. Condicionado aos *trials* observados, o processo atribui a cada combinação x ainda não testada uma média $\mu(x)$ e um desvio $\sigma(x)$, e o próximo *trial* é o máximo de uma função de aquisição, que equilibra exploração e explotação (Wu et al., 2019). Uma função de aquisição usual é a melhora esperada (*Expected Improvement*, EI) sobre o melhor valor observado até então, f^*:

$$EI(x) = \mathbb{E}\left[\max\left(f(x) - f^*,\, 0\right)\right] = \left(\mu(x) - f^*\right)\Phi(z) + \sigma(x)\,\phi(z) \tag{16}$$

onde $z = (\mu(x) - f^*)/\sigma(x)$, e Φ e ϕ são a função de distribuição acumulada e a densidade da distribuição normal padrão (Wu et al., 2019). O primeiro termo favorece a explotação, onde a

média prevista supera f^*, e o segundo, a exploração, onde a incerteza é grande. A biblioteca Optuna (Akiba et al., 2019), usada neste trabalho, implementa essa forma com núcleo de Matérn 5/2 e com o logaritmo da melhora esperada, numericamente mais estável; os primeiros *trials* são sorteados, para que o processo gaussiano tenha pontos de partida.

A seleção deve usar exclusivamente o conjunto de validação, pois escolher pelo teste o transforma em parte do ajuste. Como cada *trial* é um treino, o custo pode ser reduzido avaliando os candidatos com poucas épocas e reservando o treino longo aos mais promissores, como fazem os agendadores ASHA e RASDA (seção 2.7) (Aach, 2025). O risco é que a ordenação obtida com poucas épocas não se mantenha com um orçamento maior.

2.7 TRABALHOS RELACIONADOS

Os trabalhos relacionados foram escolhidos por tratarem de otimização: da busca de hiperparâmetros, do paralelismo no treinamento de CNNs e do próprio fluxo de classificação de HSIs que este trabalho toma como ponto de partida.

2.7.1 Protocolo de Análise

A busca foi feita no Portal de Periódicos da CAPES, com acesso fornecido pela UNIVALI, abrangendo bases como IEEE, Science Direct e ACM, e no Google Scholar, que levou a resultados de outras fontes, como Springer e Taylor & Francis. Não se trata de uma revisão sistemática: a seleção foi intencional, guiada pelos critérios a seguir.

A seleção dos trabalhos foi baseada em: (*i*) implementação de uma arquitetura de classificação de HSI; (*ii*) apresentação detalhada da arquitetura; (*iii*) implementação de um algoritmo para processamento espectral e espacial; e (*iv*) apresentação das métricas empregadas para avaliar precisão e/ou tempo de processamento. Na triagem, foram descartadas as arquiteturas que empregam mais de um modelo de ML, como uma rede convolucional encadeada a uma rede recorrente, ou um conjunto grande de algoritmos de processamento. O interesse deste trabalho está no algoritmo espacial (a segmentação) acoplado a uma rede simples, e não na combinação de muitos algoritmos para chegar ao estado da arte.

Foram selecionados três trabalhos. Apenas o de Souza (2023) atende aos quatro critérios. Os de Aach (2025) e Nandan (2024) não tratam de HSIs nem de processamento espectral-espacial (critérios *i* e *iii*) e foram incluídos pelos temas de otimização que este trabalho aplica ao fluxo de Souza (2023): a otimização de hiperparâmetros em escala, no primeiro, e o paralelismo no treinamento de CNNs, no segundo (Quadro 1).

2.7.2 Parallel and Scalable Hyperparameter Optimization for Distributed Deep Learning Methods on High-Performance Computing Systems

Aach (2025) propõe métodos para realizar Otimização de Hiperparâmetros (HPO) de forma eficiente em sistemas de Computação de Alto Desempenho (HPC) com grandes volumes de dados científicos, que vão de centenas de gigabytes a vários terabytes. O desempenho de redes neurais depende dos hiperparâmetros, que incluem parâmetros arquiteturais (número de camadas, funções de ativação, tamanho de *kernels*) e parâmetros do otimizador (taxa de aprendizado, *weight decay*, *momentum*, tamanho do lote).

O estudo investiga espaços de busca multidimensionais em três domínios. Para Visão Computacional usando ResNet50 no ImageNet, os hiperparâmetros otimizados incluem: taxa de aprendizado (log[10^{-5}, 1]), *weight decay* (log[0, 10^{-1}]), épocas de *warm-up* inicial ([1-5]), tipo de otimizador (SGD, Adam), inicialização de camadas (Kaiming, Xavier), função de ativação (ReLU, LeakyReLU, SELU, Tanh, Sigmoid) e tamanho do *kernel* de convolução ([5, 7, 9]). No domínio de Fluidodinâmica Computacional (CFD), com 8,3 TB de dados e um autoencoder convolucional, inclui-se, além dos hiperparâmetros anteriores, o *re-scaling warm-up* ([1, 2]), que ajusta gradualmente a taxa de aprendizado à medida que o número de GPUs aumenta. Para Manufatura Aditiva (AM), com SwinTransformer, o espaço adiciona: tamanho do *patch* ([2, 4]), profundidade ([1, 2, 4]), número de cabeças de atenção ([3, 6, 12, 24]) e *MLP ratio* ([1.0, 2.0, 3.0, 4.0]).

O autor desenvolve o RASDA (*Resource-Adaptive Successive Doubling Algorithm*), um agendador escalável que realoca GPUs durante a otimização, tirando recursos dos *trials* de baixo desempenho para acelerar os mais promissores. Na validação experimental, em até 1024 GPUs, o RASDA foi até $1,9\times$ mais rápido que o ASHA padrão e, no experimento CFD, chegou a um erro quadrático médio de $3,63 \times 10^{-7}$.

A Figura 15 mostra o que distingue o RASDA: enquanto o ASHA mantém constantes o lote global e a alocação de GPUs, o RASDA aumenta progressivamente os dois à medida que identifica *trials* promissores. No caso CFD, o lote global cresce de 16 para 128, e as GPUs por *trial*, de 8 para 64.

2.7.3 Comparative Analysis of TensorFlow and PyTorch for Image Classification using CNN with Parallelism Strategies

Nandan (2024) investigou como o paralelismo de dados afeta o tempo de treinamento e a acurácia de CNNs de classificação de imagens em dois *frameworks*, comparando a MirroredStrategy do TensorFlow com a DataParallel do PyTorch nos conjuntos CIFAR-10 e CIFAR-100. O modelo tem três camadas convolucionais seguidas por duas camadas densas. Cada *framework* foi testado em modo sequencial e com paralelismo de dados, e foram coletados a acurácia de

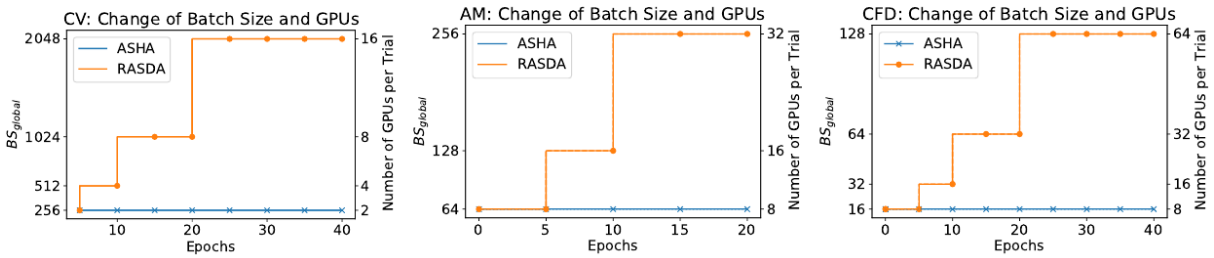


Figura 15 – Comparação entre o tamanho global do lote e o número de GPUs por *trial* no ASHA e no RASDA, nos diferentes casos de aplicação

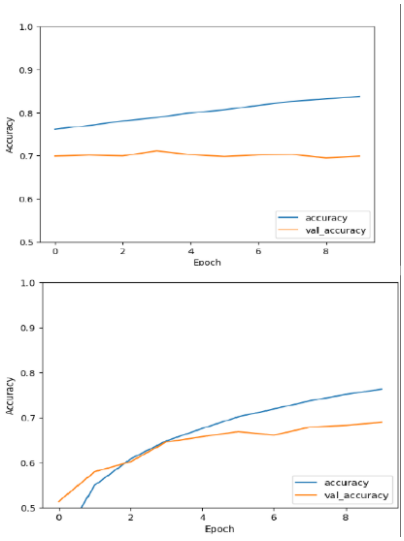
Fonte: Aach, 2025

teste, o tempo de treinamento e as curvas de aprendizado.

No CIFAR-10, o paralelismo aumentou o tempo de treinamento, o que o autor atribui à sobrecarga de comunicação: o TensorFlow passou de 84,53 s para 144,6 s, e o PyTorch, de 210,42 s para 253,2 s. Com paralelismo, o PyTorch obteve acurácia maior (0,717, contra 0,680 do TensorFlow). As curvas das Figuras 16 e 17 mostram, nos dois *frameworks*, a acurácia de treino subindo ao longo das dez épocas enquanto a de validação se estabiliza perto de 0,70, com distância crescente entre as duas; as curvas do PyTorch com e sem paralelismo são quase iguais, e as do TensorFlow diferem mais entre si. No CIFAR-100, o paralelismo reduziu os tempos de treinamento: o do TensorFlow de 224,23 s para 202,84 s, e o do PyTorch de 373,45 s para 341,4 s. A acurácia do PyTorch subiu de 0,43 para 0,45, e a do TensorFlow caiu de 0,46 para 0,42.

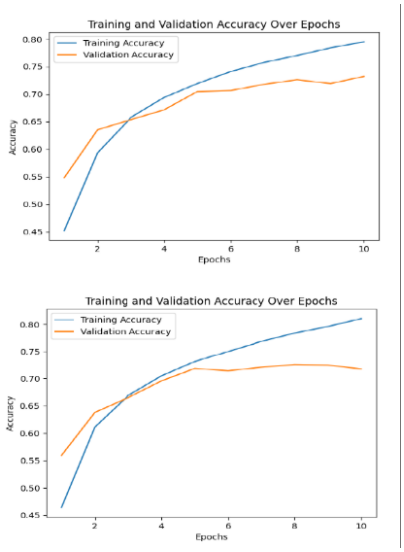
O autor associa o TensorFlow à prototipagem rápida e o PyTorch a aplicações em que a acurácia é prioritária, mas os experimentos medem apenas tempo de treinamento e acurácia no CIFAR; nada neles avalia prototipagem ou uso em produção. Do mesmo modo, o paralelismo aumentou o tempo no CIFAR-10 e o reduziu no CIFAR-100, mas os dois conjuntos têm o mesmo número e o mesmo tamanho de imagens e diferem apenas no número de classes, de modo que esses resultados não bastam para atribuir a diferença à escala do problema. O que eles mostram, e que interessa a este trabalho, é que a sobrecarga de coordenar a execução paralela pode superar o ganho.

Figura 16 – Curvas de acurácia de treino e de validação do TensorFlow com as estratégias sequencial e de paralelismo de dados no CIFAR-10



Fonte: Nandan, 2024

Figura 17 – Curvas de acurácia de treino e de validação do PyTorch com as estratégias sequencial e de paralelismo de dados no CIFAR-10



Fonte: Nandan, 2024

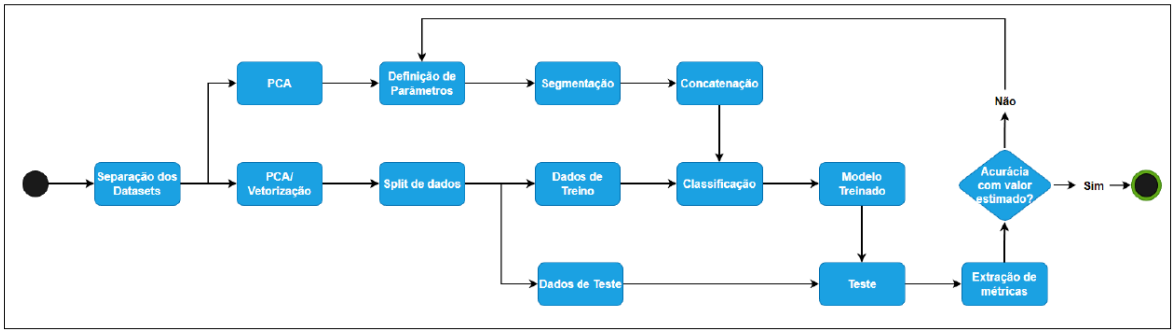
2.7.4 Avaliação de Fusão de Dados Espaciais com Espectrais para Classificação de Imagens Hiperespectrais

O trabalho de Souza (2023) é o ponto de partida deste estudo. O autor aborda a classificação de HSIs, em que há um compromisso entre complexidade computacional e acurácia, e observa que os métodos tradicionais se concentram nas características espectrais e subutilizam o contexto espacial, que pode melhorar a distinção entre classes. Os objetivos são avaliar o impacto do pré-processamento espacial na classificação por aprendizado de máquina, integrar a

segmentação SEEDS a uma CNN 1D por concatenação, comparar o resultado com modelos de base e com o estado da arte, e quantificar o custo computacional da abordagem.

A metodologia propõe uma arquitetura híbrida que combina uma CNN 1D, para a classificação espectral, com o SEEDS, para a segmentação espacial. A Figura 18 mostra o fluxo completo, da separação dos conjuntos de dados à extração das métricas de avaliação. O processamento tem dois ramos independentes, executados em sequência: o ramo superior realiza PCA, definição de parâmetros, segmentação e concatenação das características espaciais; o ramo inferior executa PCA e vetorização, divisão dos dados de treino e classificação pela CNN 1D. Na implementação original, a CNN 1D tem quatro camadas convolucionais, com 16, 32, 64 e 64 filtros e sem *pooling*, seguidas por duas camadas densas, de 128 e 64 neurônios, e pela camada de saída; a rede usada neste trabalho, a prevista no Capítulo 3, acrescenta *max-pooling*, tem 128 filtros na última convolução e densas de 256 e 128 neurônios (Tabela 1). A fusão é feita por uma camada de concatenação, que junta às características extraídas pelas convoluções o identificador do superpixel do pixel. Os hiperparâmetros da rede e do SEEDS foram ajustados manualmente: o autor relata ter alterado, por conjunto de dados, a paciência da parada antecipada e outros parâmetros de treino ao buscar o melhor resultado (Souza, 2023).

Figura 18 – Fluxograma da metodologia proposta, com os ramos espacial e espectral, a fusão via concatenação, a classificação e a validação iterativa do modelo



Fonte: Souza, 2023

A validação experimental foi conduzida em três cenas: Indian Pines (21.025 pixels, 10.249 rotulados, 16 classes), Salinas Valley (111.104 pixels, 54.129 rotulados, 16 classes) e University of Pavia (207.400 pixels, 42.776 rotulados, 9 classes). As métricas avaliadas incluem Acurácia Geral (OA), Acurácia Média (AA), coeficiente Kappa, tempo de inferência e uso de memória. Com a fusão SEEDS, o autor reporta OA de 0,9142, AA de 0,8956 e Kappa de 0,9019 em Indian Pines, OA de 0,9928 em University of Pavia e OA de 0,9965 em Salinas Valley; em Indian Pines, a fusão acrescentou 8,33, 8,79 e 9,47 pontos percentuais de OA, AA e Kappa à mesma rede sem a informação espacial. Essa configuração, com a fusão SEEDS, é a referência contra a qual os ganhos deste trabalho são medidos: os hiperparâmetros de treino e

os parâmetros do SEEDS do autor formam a linha de base, executada com a CNN prevista no Capítulo 3, e a rede do autor é treinada nas mesmas condições apenas para a comparação com os valores reportados (Tabela 2). O fluxograma da Figura 18 mostra ainda o processo iterativo de treinamento, teste e extração de métricas, com validação baseada em um valor estimado de acurácia.

2.7.5 Análise Comparativa dos Trabalhos Relacionados

O Quadro 1 compara os três trabalhos com este. Os de Aach (2025) e Nandan (2024) tratam de otimização, de hiperparâmetros em escala e de paralelismo no treinamento, respectivamente, mas em problemas de visão computacional, fluidodinâmica e manufatura aditiva, sem HSIs nem segmentação. O de Souza (2023) trata de HSIs com fusão espectral-espacial, mas executa as etapas em sequência, ajusta os hiperparâmetros à mão e usa um único segmentador. Nenhum dos três combina, no mesmo fluxo de classificação de HSIs, a busca automática de hiperparâmetros da rede e do segmentador, a comparação de segmentadores sob a mesma otimização e o paralelismo entre as etapas; é essa a lacuna que este trabalho ocupa.

Quadro 1 – Comparação dos trabalhos relacionados

Característica	(Aach, 2025)	(Nandan, 2024)	(Souza, 2023)	Este Trabalho
Arquitetura de IA	ResNet50 (ImageNet), autoencoder convolucional (CFD) e Swin-Transformer (AM)	CNN customizada	CNN 1D (16 a 64 filtros, sem *pooling*)	CNN 1D do projeto (16 a 128 filtros, com *max-pooling*) e fusão espectral-espacial
Processamento Paralelo	Sim. RASDA	Sim. Paralelismo de dados (MirroredStrategy e DataParallel)	Não. Ramos independentes, executados em sequência	Sim. Threads entre os ramos do pré-processamento e na fusão concorrente; threads e processos na partição da cena
Segmentação	Não aplicado	Não aplicado	Sim. SEEDS ajustado manualmente	Sim. SEEDS, LSC e Watershed, com o mesmo número de regiões e com a configuração otimizada junto com a da CNN
HPO	Sim. ASHA e RASDA	Não abordada	Não. Ajuste manual por conjunto de dados	Sim. Busca em grade e Otimização Bayesiana com processo gaussiano, para a CNN e o segmentador
Métricas	Val MSE, Test MSE, Runtime, Relative Error	Test Accuracy, Training Time, Val Accuracy	OA, AA, Kappa, Tempo, Memória	OA, AA, Kappa, Tempo, Memória por processo, *Speedup*
Datasets	ImageNet, CFD, RAISE-LPBF	CIFAR-10, CIFAR-100	Indian Pines, Salinas, Pavia	Indian Pines, Salinas Valley, University of Pavia
Tecnologias	Python, PyTorch, Horovod, DeepSpeed, Ray Tune, CUDA 11.5, NVIDIA DALI	Python, TensorFlow, PyTorch, Keras, CUDA	Python, TensorFlow, Keras, scikit-learn, OpenCV, Google Colab	Python, PyTorch, scikit-learn, OpenCV, Optuna

Fonte: Elaborado pelo autor

3 PROJETO

Neste trabalho será proposta a implementação de estratégias de otimização de classificação de imagens hiperespectrais utilizando CNNs 1D combinadas com técnicas de segmentação espacial. A partir de arquiteturas e algoritmos encontrados na literatura, procura-se fazer uma análise com base nas métricas elencadas neste projeto. O capítulo está organizado da seguinte forma: inicialmente é apresentada a visão geral do sistema proposto. Na sequência, são dispostas as premissas que norteiam o desenvolvimento. Posteriormente, são descritos os materiais e métodos utilizados e os requisitos levantados, seguidos do detalhamento da arquitetura do sistema e dos algoritmos. Ao fim, são apresentados o plano de verificação e a análise de riscos.

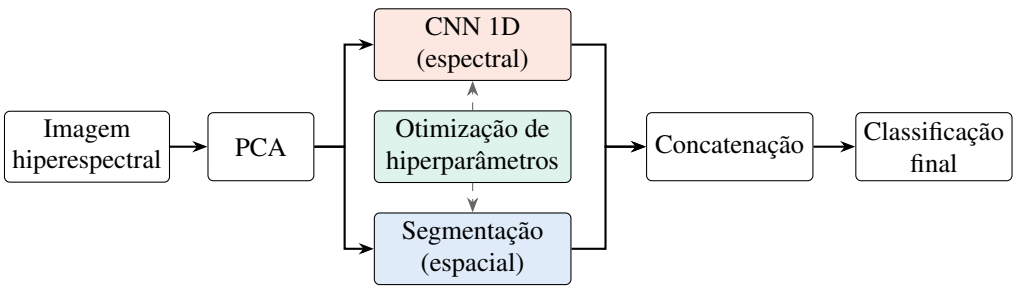
3.1 VISÃO GERAL

O projeto visa paralelizar o processamento da arquitetura de Souza (2023) e automatizar o ajuste de hiperparâmetros tanto do modelo de classificação quanto dos algoritmos de segmentação. As técnicas específicas são detalhadas na subseção 3.4.2.

Para a avaliação da eficácia e do custo computacional das otimizações propostas, serão utilizados três datasets de referência na área: Indian Pines, Salinas Valley e University of Pavia. Os detalhes da campanha de experimentos são abordados na subseção 3.5.1.

A Figura 19 sumariza a relação entre os principais componentes do sistema proposto. A arquitetura híbrida processa simultaneamente as características espectrais via CNN 1D e as características espaciais via algoritmos de segmentação, realizando a fusão dos dados através de uma camada de concatenação. O processo de otimização de hiperparâmetros é controlado por algoritmos automatizados que ajustam tanto os parâmetros da rede neural quanto os parâmetros de segmentação.

Figura 19 – Principais componentes do projeto



Fonte: Elaborada pelo autor

3.2 PREMISSAS

O trabalho parte do pressuposto de que a arquitetura base proposta por Souza (2023), que combina CNN 1D com segmentação SEEDS, apresenta gargalos de processamento que podem ser mitigados por técnicas de paralelização e otimização automática de hiperparâmetros. Especificamente, assume-se que o processamento sequencial das etapas de segmentação e classificação pode ser acelerado executando-as de forma concorrente; a fração do tempo de execução ocupada por cada etapa será medida na campanha experimental.

Será considerado que os três datasets selecionados (Indian Pines, com 16 classes e 10.249 pixels rotulados de um total de 21.025; Salinas Valley, com 16 classes e 54.129 rotulados de 111.104; e University of Pavia, com 9 classes e 42.776 rotulados de 207.400) são representativos o suficiente para validar as melhorias propostas e permitir a comparação direta com Souza (2023), que utilizou as mesmas cenas.

Presume-se que a separação de 85% das amostras para treinamento e 15% para validação e teste, divididos igualmente entre os dois, conforme adotado por Souza (2023) e Viel et al. (2023), é adequada para avaliar o desempenho dos modelos.

Quanto à otimização de hiperparâmetros, será assumido que técnicas de busca automatizada como Grid Search e Otimização Bayesiana são capazes de encontrar configurações próximas ao ótimo global dentro de um espaço de busca adequadamente definido.

Em relação ao processamento paralelo, presume-se que a paralelização das etapas de segmentação e classificação espectral pode reduzir significativamente o tempo total de processamento, dado que essas operações apresentam independência de dados durante parte do fluxo de execução.

Sob essas premissas, os experimentos controlados medirão os ganhos relativos de cada otimização proposta sobre a configuração de Souza (2023).

3.3 METODOLOGIA

3.3.1 Materiais

3.3.1.1 Ambiente Computacional e Ferramentas

O desenvolvimento e experimentação serão realizados utilizando a linguagem Python com as seguintes bibliotecas principais: TensorFlow/Keras ou PyTorch para implementação da CNN 1D, Scikit-Learn para pré-processamento (PCA) e métricas, OpenCV para manipulação de imagens, e Ray Tune para otimização de hiperparâmetros.

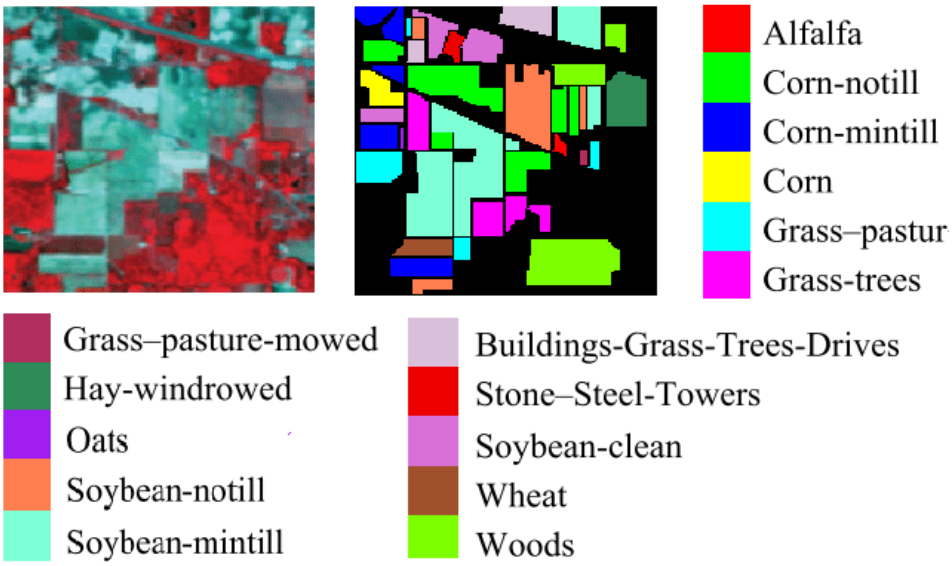
Os experimentos computacionalmente intensivos serão executados em ambiente de computação de alto desempenho (HPC) ou Google Colab Pro com acesso a GPUs. Para experimentos preliminares e desenvolvimento, será utilizado ambiente local com CPU de múltiplos núcleos.

3.3.1.2 Datasets Hiperespectrais

Serão utilizados três datasets públicos de imagens hiperespectrais, os mesmos avaliados por Souza (2023) (Quadro 1), ilustrados nas Figuras 20, 21 e 22.

O dataset Indian Pines (Figura 20) representa uma área agrícola capturada em 1992 pelo sensor AVIRIS no noroeste de Indiana. A imagem possui 145×145 pixels, com resolução espacial de 20 metros por pixel; dos seus 21.025 pixels, 10.249 são rotulados em 16 classes de cobertura do solo. Das 224 bandas originais capturadas pelo sensor, foram utilizadas 200 bandas espectrais na faixa de 400 a 2500 nanômetros com intervalos de 10 nanômetros, após a remoção de 4 bandas nulas e 20 bandas comprometidas pela absorção atmosférica de água. A visualização apresenta a composição em falsa-cor à esquerda, o mapa de verdade terrestre ao centro e a legenda com as 16 classes, que incluem diferentes tipos de culturas agrícolas (alfafa, milho, soja, trigo), pastagens, matas e áreas construídas (Laban et al., 2020).

Figura 20 – Indian Pines: composição falsa-cor, mapa de verdade terrestre e legenda

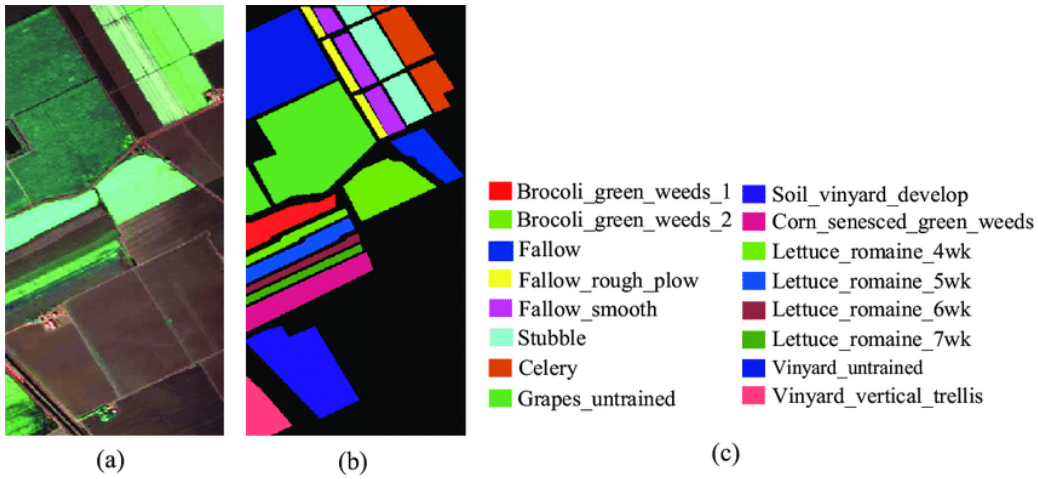


Fonte: Laban et al., 2020

O dataset Salinas Valley (Figura 21) foi capturado pelo sensor AVIRIS na região do Vale de Salinas, Califórnia, com resolução espacial de 3,7 metros por pixel. A imagem possui 512×217 pixels e originalmente continha 224 bandas espectrais; após a remoção de 20 bandas afetadas pela absorção de água, restaram 204 bandas utilizáveis para análise. Dos 111.104 pixels da imagem, 54.129 são rotulados em 16 classes, predominantemente culturas agrícolas e tipos de

cobertura do solo. A imagem (a) mostra a composição falsa-cor da área agrícola, (b) apresenta o mapa de verdade terrestre e (c) exibe a legenda com as 16 classes, que incluem duas classes de brócolis, alface romana em diferentes estágios, vinhedos, solo em pousio e outras culturas (Li et al., 2022).

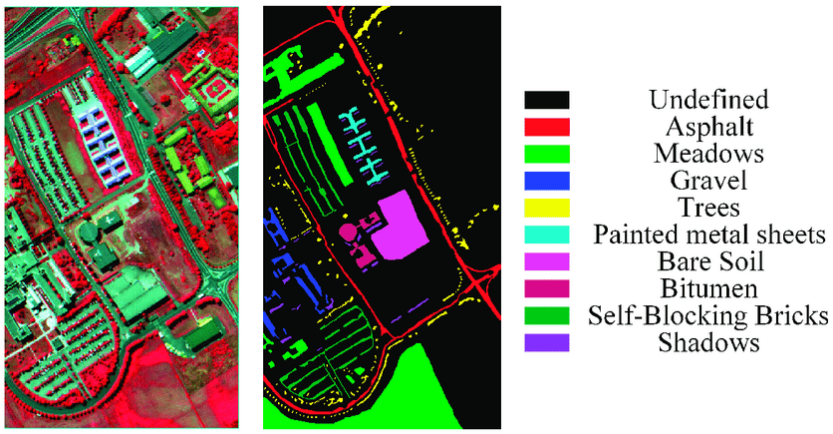
Figura 21 – Salinas Valley: (a) falsa-cor, (b) verdade terrestre, (c) legenda



Fonte: Li et al., 2022

Por fim, o dataset University of Pavia (Figura 22) representa uma área urbana da Universidade de Pavia, Itália, capturada em 2003 pelo sensor ROSIS. A imagem possui 610×340 pixels, com resolução espacial de 1,3 metro por pixel. Das 115 bandas espectrais originais, capturadas na faixa de comprimento de onda entre 0,43 e 0,86 micrômetro (430 a 860 nanômetros), foram utilizadas 103 bandas, após a remoção de 12 bandas contaminadas por ruído. Dos 207.400 pixels da imagem, 42.776 são rotulados em 9 classes de cobertura urbana. A visualização mostra em (a) a composição falsa-cor da área universitária, em (b) o mapa de verdade terrestre das estruturas urbanas e a legenda com as 9 classes que incluem asfalto, prados, cascalho, árvores, chapas metálicas pintadas, solo exposto, betume, tijolos autoblocantes e sombras (Qi et al., 2019).

Figura 22 – University of Pavia: (a) falsa-cor, (b) verdade terrestre e legenda



Fonte: Qi et al., 2019

Para versionamento de código será utilizado Git, e para documentação e reprodutibilidade dos experimentos será mantido registro detalhado de todas as configurações utilizadas através de arquivos de configuração e notebooks Jupyter.

3.3.2 Métodos

Serão utilizadas as seguintes técnicas e estratégias de otimização: paralelização do processamento espacial e espectral, otimização automática de hiperparâmetros (Grid Search e Otimização Bayesiana), comparação de diferentes algoritmos de segmentação como SEEDS, LSC e Watershed, e redução dimensional via PCA. Cada técnica é detalhada na subseção 3.4.2.

A coleta de métricas será realizada utilizando as ferramentas nativas do TensorFlow/Keras ou do PyTorch para monitoramento do treinamento, bibliotecas de *profiling* do Python para medição do tempo de execução e utilitários do sistema para medição do uso de memória. As métricas principais são a Acurácia Geral (OA) e a Acurácia Média (AA) (Congalton, 1991), o coeficiente Kappa (Cohen, 1960), os tempos de treinamento e de inferência, o uso de memória e o *speedup* obtido com a paralelização.

A fim de promover a reutilização de código e facilitar a reprodução dos experimentos, será implementado um *framework* modular que abstrai os componentes principais do sistema: módulo de pré-processamento, módulo de segmentação, módulo de classificação CNN, módulo de fusão e módulo de avaliação. A interação entre esses módulos é detalhada na subseção 3.4.1.

3.4 ANÁLISE DE REQUISITOS

Quadro 2 – Requisitos funcionais

Requisito	Descrição
RF01	A arquitetura deve classificar HSIs utilizando CNN 1D para extração de características espectrais
RF02	A arquitetura deve implementar algoritmos de segmentação espacial
RF03	A arquitetura deve realizar fusão de dados espectrais e espaciais
RF04	O sistema deve implementar otimização automática de hiperparâmetros
RF05	O sistema deve produzir métricas de desempenho e custo computacional
RF06	O sistema deve implementar paralelização do processamento espacial e espectral

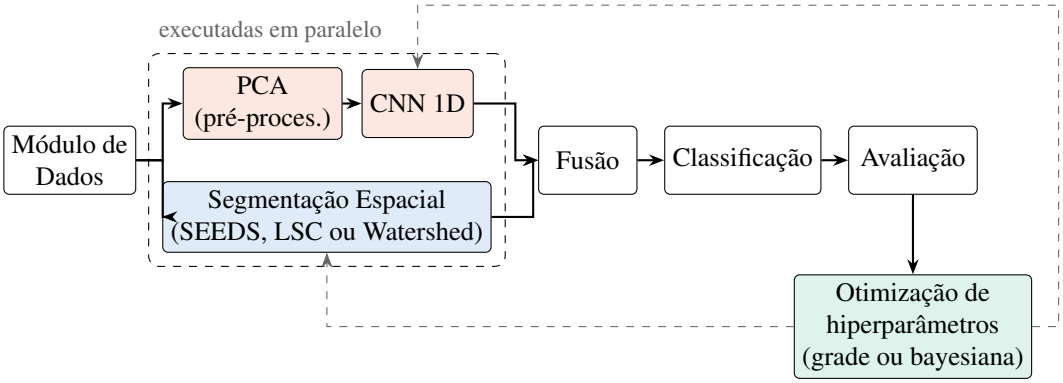
Quadro 3 – Requisitos não funcionais

Requisito	Descrição
RNF01	O classificador espectral deve ser uma CNN 1D baseada no modelo de Souza (2023) e Viel et al. (2023)
RNF02	Toda a arquitetura deve ser desenvolvida na linguagem de programação Python
RNF03	Para treinar, testar e validar o modelo deve ser utilizada a plataforma Google Colab ou ambiente HPC com GPU
RNF04	A fusão de dados deve ser realizada por concatenação dos segmentos espaciais com as características espectrais na CNN
RNF05	A otimização automática de hiperparâmetros deve melhorar as métricas de acurácia (OA, AA, Kappa) em comparação ao ajuste manual
RNF06	O sistema deve produzir resultados reprodutíveis através de registro detalhado de configurações
RNF07	A paralelização deve alcançar redução mensurável no tempo de processamento em relação à implementação sequencial

3.4.1 Arquitetura do Sistema

A arquitetura proposta segue uma organização modular que permite experimentação com diferentes configurações. A Figura 23 apresenta o diagrama de blocos detalhado do sistema completo.

Figura 23 – Arquitetura modular do sistema



Fonte: Elaborada pelo autor

A arquitetura proposta organiza-se em módulos funcionais que operam de forma integrada. O Módulo de Dados é responsável pelo carregamento e preparação dos datasets, abstraindo as diferenças entre formatos distintos e fornecendo uma interface unificada para o sistema, além de realizar a separação dos conjuntos de treino, validação e teste com base em Souza (2023) e Viel et al. (2023).

O processamento dos dados ocorre em duas frentes paralelas. De um lado, o Módulo de Pré-processamento aplica a técnica de PCA para redução dimensional, transformando o hipercubo espectral original em um conjunto reduzido de componentes principais cujo número pode ser configurado conforme necessário. Do outro, o Módulo de Segmentação Espacial implementa os algoritmos SEEDS, LSC e Watershed de forma intercambiável, produzindo mapas de superpixels que capturam a estrutura espacial das imagens.

As características espectrais resultantes do pré-processamento são então alimentadas ao Módulo CNN 1D, que implementa uma rede neural convolucional unidimensional com arquitetura configurável, composta por camadas de convolução 1D seguidas de camadas densas totalmente conectadas. Este módulo extrai características discriminativas de cada pixel a partir de sua assinatura espectral.

O Módulo de Fusão realiza a concatenação entre as características espectrais extraídas pela CNN 1D e os rótulos de superpixels provenientes da segmentação espacial, unindo as informações complementares em um único vetor de características. Essas características fusionadas são então processadas pelo Módulo de Classificação, que realiza a predição final das classes para cada pixel da imagem.

O Módulo de Avaliação quantifica o desempenho do sistema através de métricas de acurácia e custo computacional. Por fim, o Módulo de Otimização de Hiperparâmetros coordena

a busca automática dos melhores parâmetros tanto para os algoritmos de segmentação quanto para a arquitetura da CNN, utilizando técnicas como Grid Search ou Otimização Bayesiana para maximizar o desempenho global do sistema.

3.4.2 Algoritmos e Técnicas

A otimização e a classificação de imagens hiperespectrais combinarão técnicas de aprendizado profundo, processamento de imagens e otimização automática, descritas a seguir.

3.4.2.1 Redução Dimensional via PCA

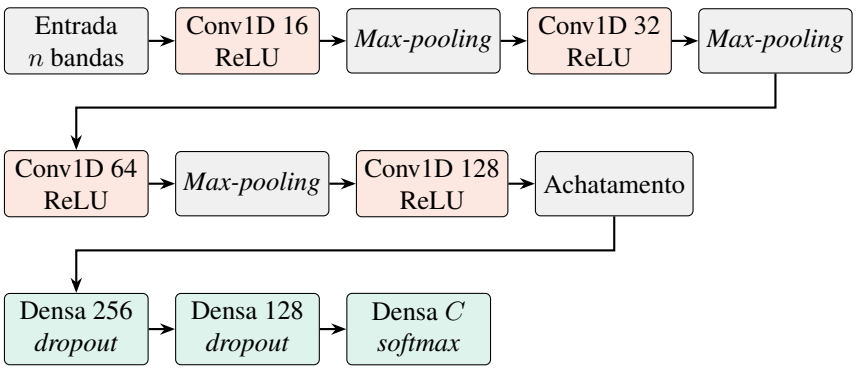
A técnica de Análise de Componentes Principais (PCA) será aplicada para reduzir a dimensionalidade espectral das imagens hiperespectrais. Dado um conjunto de dados com n bandas espectrais, o PCA identifica as k componentes principais (onde $k < n$) que capturam a maior variância dos dados (Jolliffe, 2002).

3.4.2.2 Rede Neural Convolucional 1D (CNN 1D)

A arquitetura da CNN 1D é projetada para processar as características espectrais unidimensionais de cada pixel. A arquitetura base consiste em:

- 4 camadas convolucionais 1D com 16, 32, 64 e 128 filtros respectivamente
- Função de ativação ReLU após cada camada convolucional
- Operações de *max-pooling* para redução dimensional
- 3 camadas densas (totalmente conectadas) com *dropout* para regularização (Srivastava et al., 2014)
- Camada de saída com ativação *softmax* para classificação multiclasse

Figura 24 – Arquitetura da CNN 1D prevista no projeto



Fonte: Elaborada pelo autor. C: número de classes da cena

Os hiperparâmetros da rede (número de filtros, tamanho dos *kernels*, taxa de *dropout*, taxa de aprendizado, otimizador) serão otimizados automaticamente pelo módulo de HPO.

3.4.2.3 Algoritmos de Segmentação Espacial

Três algoritmos de segmentação serão implementados e comparados:

- **SEEDS** (*Superpixels Extracted via Energy-Driven Sampling*): parte de uma grade regular de superpixels e refina as fronteiras por *hill-climbing*, transferindo blocos de pixels e, depois, pixels individuais entre superpixels vizinhos sempre que a troca aumenta uma função de energia que favorece a homogeneidade de cor de cada superpixel e fronteiras regulares. Os principais parâmetros ajustáveis são o número desejado de superpixels, o número de iterações e o número de níveis de blocos usados no refinamento (Van den Bergh et al., 2013).
- **LSC** (*Linear Spectral Clustering*): realiza agrupamento espectral considerando tanto a cor quanto a posição espacial dos pixels. Os principais parâmetros são o número de superpixels, o peso da regularização espacial e o número de iterações (Li; Chen, 2015).
- **Watershed**: algoritmo clássico de segmentação baseado em morfologia matemática, que trata a imagem como uma superfície topográfica. Os parâmetros são o método de marcação das sementes e o nível de compactação (Najman; Schmitt, 1996).

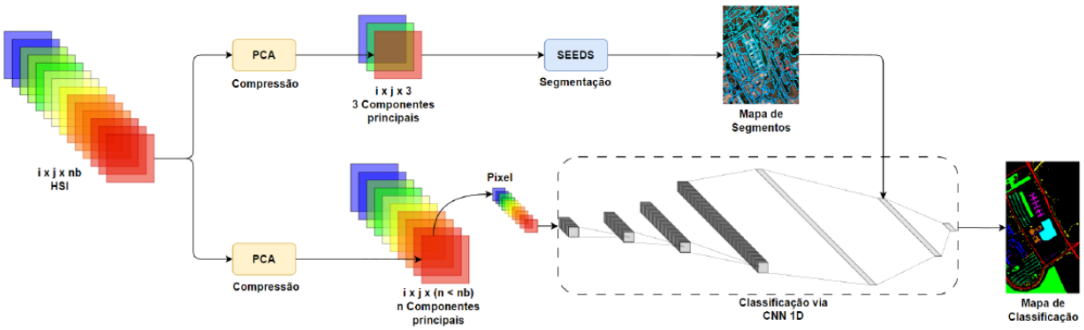
Cada algoritmo será executado sobre as três primeiras componentes principais obtidas via PCA, gerando um mapa de superpixels que identifica regiões espacialmente coerentes na imagem.

3.4.2.4 Fusão de Características por Concatenação

A fusão das características espectrais (extraídas pela CNN 1D) com as características espaciais (rótulos de superpixels) é realizada por uma camada de concatenação, como em Souza (2023), que concatena o identificador do superpixel como um único valor numérico. Neste projeto, está prevista a codificação do identificador em um vetor *one-hot*. Para cada pixel:

1. A CNN 1D extrai um vetor de características espectrais $\mathbf{f}_{\text{esp}} \in \mathbb{R}^d$;
2. O algoritmo de segmentação atribui um rótulo de superpixel $l \in \{1, 2, \ldots, S\}$, em que S é o número de superpixels;
3. O rótulo é codificado como vetor *one-hot* $\mathbf{f}_{\text{sp}} \in \mathbb{R}^S$;
4. As características são concatenadas: $\mathbf{f} = [\mathbf{f}_{\text{esp}}, \mathbf{f}_{\text{sp}}]$.

Figura 25 – Processo de fusão por concatenação



Fonte: Souza, 2023

O vetor de características fusionadas é então alimentado a camadas densas adicionais que realizam a classificação final.

3.4.2.5 Paralelização do Processamento

Para acelerar o fluxo de processamento, a seguinte estratégia de paralelização será implementada, explorando a independência de dados entre as etapas de segmentação e classificação:

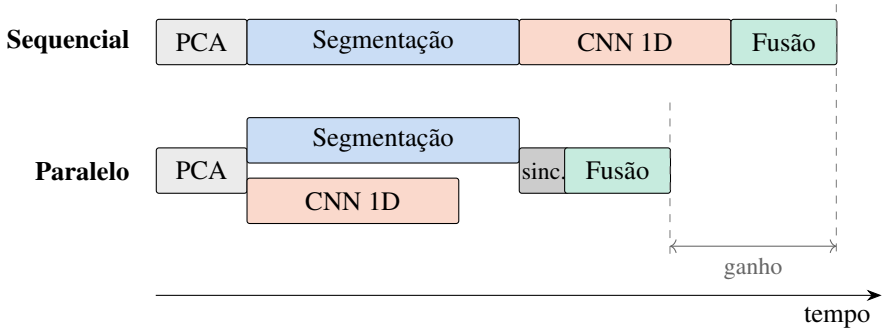
Paralelização Inter-Etapas: A segmentação espacial e a extração de características espectrais pela CNN são processos independentes que podem ser executados simultaneamente. Esta independência existe porque:

- A segmentação opera sobre as componentes principais do PCA para gerar o mapa de superpixels, sem necessitar das características extraídas pela CNN.
- A CNN 1D processa os vetores espectrais de cada pixel de forma independente da segmentação.

Utilizando a biblioteca `multiprocessing` do Python para a segmentação (operação CPU-bound) e execução em GPU para a CNN (via CUDA cores), estas etapas são executadas em paralelo. A sincronização ocorre apenas no ponto de fusão, onde os resultados de ambas as operações são necessários.

Paralelização Intra-Etapas: Dentro da CNN 1D, o processamento em lotes permite aproveitar o paralelismo da GPU. As operações de convolução são intrinsecamente paralelizáveis porque cada elemento do mapa de características de saída pode ser calculado de forma independente dos demais, sem dependência de dados dentro da mesma camada. Esta propriedade permite que os CUDA cores da GPU calculem simultaneamente diferentes elementos de saída. Para a segmentação, algumas implementações do OpenCV, como a do LSC, dividem o cálculo entre múltiplas threads.

Figura 26 – Estratégia de paralelização: fluxo sequencial versus fluxo paralelo



Fonte: Elaborada pelo autor

A Figura 26 ilustra a diferença entre os dois fluxos de execução. No fluxo sequencial, após o PCA, a segmentação é executada primeiro, seguida pela CNN 1D, e então a fusão. No fluxo paralelo proposto, após o PCA, dois fluxos de execução são iniciados concorrentemente: um executa a segmentação espacial enquanto o outro executa a CNN 1D. O tempo esperado do fluxo paralelo é o maior entre os tempos das duas operações, acrescido da sobrecarga de coordenação (seção 2.4), de modo que o *speedup* esperado é $(T_{seg} + T_{cnn})/[\max(T_{seg}, T_{cnn}) + T_{overhead}]$, limitado pelo teto apresentado na seção 2.5. Por exemplo, se a segmentação levar 1 s e a CNN 3 s, o *speedup* não passará de $4/3 \approx 1{,}33$; ele só se aproxima de 2 quando as duas etapas têm durações semelhantes.

3.4.2.6 Otimização Automática de Hiperparâmetros

Duas abordagens de otimização serão implementadas:

Grid Search: Busca exaustiva em um espaço discreto de hiperparâmetros. Para cada combinação possível de parâmetros, um modelo é treinado e avaliado. Apesar de custoso computacionalmente, garante exploração completa do espaço de busca definido (Liashchynskyi; Liashchynskyi, 2019).

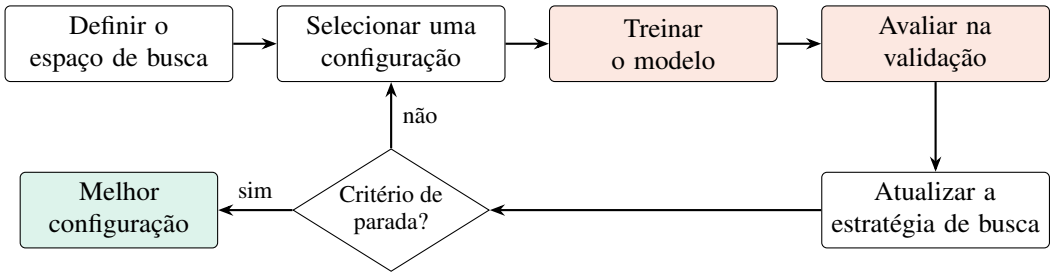
Otimização Bayesiana: Abordagem mais eficiente que modela a função objetivo (acurácia de validação) como um processo Gaussiano e utiliza uma função de aquisição para decidir quais configurações testar a seguir, balanceando exploração e explotação (Wu et al., 2019).

Os hiperparâmetros a serem otimizados incluem:

- **Para a CNN 1D**: taxa de aprendizado, tamanho do lote, número de épocas, taxa de *dropout*, número de filtros por camada, tamanho dos *kernels* e otimizador (Adam, SGD, RMSprop);

- **Para a segmentação**: número de superpixels, número de iterações e parâmetros específicos de cada algoritmo;
- **Para o PCA**: número de componentes principais.

Figura 27 – Fluxo da otimização de hiperparâmetros



Fonte: Elaborada pelo autor

3.5 PLANO DE VERIFICAÇÃO

O plano de verificação estabelece um protocolo para validar o cumprimento dos requisitos estabelecidos e avaliar a eficácia das técnicas implementadas. A validação será conduzida através de uma campanha experimental, seguida de análise dos resultados obtidos.

3.5.1 Campanha Experimental

A campanha experimental será organizada de modo a garantir a comparabilidade dos resultados e permitir a análise incremental dos ganhos obtidos em cada etapa de otimização. A estratégia segue uma abordagem progressiva, em que cada experimento parte dos resultados do anterior, permitindo isolar e quantificar o impacto individual de cada técnica implementada.

Para cada dataset (Indian Pines, Salinas Valley e University of Pavia), serão realizados cinco experimentos principais, conforme detalhado no Quadro 4, o primeiro dos quais estabelece a linha de base.

Quadro 4 – Descrição detalhada dos experimentos

Experimento	Descrição
Exp. 1 - Baseline	Reprodução da arquitetura original sem otimizações (CNN 1D + SEEDS com parâmetros fixos, processamento sequencial) para estabelecer linha de base
Exp. 2 - Otimização de Segmentação	Teste dos três algoritmos de segmentação (SEEDS, LSC, Watershed) com otimização automática de seus parâmetros, mantendo CNN fixa
Exp. 3 - Otimização de CNN	Otimização dos hiperparâmetros da CNN 1D mantendo segmentação fixa
Exp. 4 - Otimização Conjunta	Otimização simultânea dos hiperparâmetros de segmentação e CNN
Exp. 5 - Paralelização	Aplicação da estratégia de paralelização sobre a melhor configuração identificada

Para cada experimento, serão realizadas 5 execuções independentes com diferentes sementes aleatórias para avaliar a variabilidade dos resultados. As métricas coletadas serão:

- Acurácia Geral (OA), Acurácia Média (AA), coeficiente Kappa
- Tempo de pré-processamento (PCA + Segmentação)
- Tempo de treinamento da CNN
- Tempo de inferência (total e por pixel)
- Uso de memória (pico durante treinamento e inferência)
- Número de parâmetros do modelo
- *Speedup* obtido com paralelização

Os resultados serão organizados conforme Quadro 5.

Quadro 5 – Planejamento de experimentos

Experimento	Dataset	Segmentação	HPO CNN	HPO Seg.	Paralelo
Exp1 - Baseline	IP, SV, UP	SEEDS	-	-	-
Exp2 - Otim. Seg.	IP, SV, UP	SEEDS, LSC, WS	-	X	-
Exp3 - Otim. CNN	IP, SV, UP	SEEDS	X	-	-
Exp4 - Otim. Conjunta	IP, SV, UP	Melhor	X	X	-
Exp5 - Paralelização	IP, SV, UP	Melhor	X	X	X

3.5.2 Análise Estatística dos Resultados

Para cada configuração, serão calculados a média e o desvio padrão de cada métrica entre as execuções com sementes diferentes, e a comparação entre configurações utilizará testes de significância estatística. O critério efetivamente adotado na execução, que substitui os testes por uma comparação com o desvio entre sementes, está registrado na seção 4.6.

3.5.3 Validação dos Requisitos

A validação dos requisitos funcionais e não funcionais estabelecidos na seção 3.4 será conduzida através de procedimentos específicos para cada requisito, conforme detalhado no Quadro 6 e no Quadro 7. Para cada requisito, são especificados os aspectos a serem validados, os métodos de validação a serem empregados e os critérios objetivos de sucesso.

Quadro 6 – Plano de verificação dos requisitos funcionais

Req.	Aspecto a Validar	Método de Validação	Critério de Sucesso
RF01	Implementação correta da CNN 1D para extração de características espectrais	Testes verificando dimensões dos tensores em cada camada	Dimensões de saída de cada camada iguais às previstas na arquitetura
RF02	Implementação correta dos algoritmos de segmentação espacial	Testes verificando geração de mapas de superpixels	Mapas de segmentação gerados corretamente para os três algoritmos
RF03	Sistema de fusão por concatenação funcionando	Verificação das dimensões das características fusionadas correspondendo à soma das dimensões espectrais e espaciais	Fusão executada corretamente e classificação com fusão superior à classificação puramente espectral
RF04	Sistema de otimização automática de hiperparâmetros	Comparação entre algoritmos, verificando convergência e qualidade das configurações encontradas	OA da melhor configuração encontrada superior à da configuração de Souza (2023), nas mesmas condições de treino
RF05	Produção de métricas de desempenho e custo computacional	Coleta e registro de todas as métricas especificadas	Todas as métricas coletadas e registradas para análise comparativa
RF06	Eficácia da paralelização do processamento	Medição do tempo de execução com e sem paralelização, cálculo do *speedup* obtido	*Speedup* maior que 1 em relação ao fluxo sequencial, na mesma máquina

Quadro 7 – Plano de verificação dos requisitos não funcionais

Req.	Aspecto a Validar	Método de Validação	Critério de Sucesso
RNF01	Conformidade da arquitetura CNN 1D com o modelo base	Comparação estrutural com a arquitetura proposta por (Souza, 2023) e (Viel et al., 2023)	Arquitetura implementada segue o modelo base especificado
RNF02	Desenvolvimento em Python	Verificação de que todos os módulos foram desenvolvidos em Python	Todos os módulos implementados em Python
RNF03	Utilização de plataforma adequada para treinamento	Confirmação de uso de Google Colab ou ambiente HPC com GPU	Experimentos executados em ambiente com aceleração por GPU
RNF04	Fusão por concatenação implementada corretamente	Verificação de que características espectrais e espaciais são concatenadas	Fusão realizada por concatenação de vetores
RNF05	Melhoria das métricas com otimização automática	Comparação das métricas OA, AA e Kappa entre configurações otimizadas automaticamente e ajuste manual	Métricas de acurácia superiores com otimização automática
RNF06	Reprodutibilidade dos resultados	Execução de múltiplas rodadas com mesmas configurações e análise de variabilidade	Desvio padrão das métricas inferior a 2% entre execuções independentes
RNF07	Ganho de desempenho com paralelização	Medição quantitativa do *speedup* obtido	*Speedup* maior que 1 em relação à implementação sequencial, na mesma máquina

O Quadro 8 estabelece, para cada dataset, as metas de acurácia que o sistema otimizado deve atingir ou superar para ser considerado bem-sucedido. As metas são definidas neste projeto, e a comparação de referência é com os valores reportados por Souza (2023) nas mesmas cenas: OA de 0,9142 em Indian Pines, 0,9965 em Salinas Valley e 0,9928 em University of Pavia. A meta de OA de Indian Pines fica, portanto, acima do valor reportado, e as de Salinas Valley e University of Pavia, abaixo.

Quadro 8 – Validação de desempenho nos datasets de referência

Dataset	Aspecto a Validar	Método de Validação	Meta
Indian Pines	Métricas OA, AA e Kappa iguais ou superiores às metas	Comparação com as metas e com os valores reportados por Souza (2023)	$OA \geq 95\%$, $AA \geq 93\%$, $Kappa \geq 0{,}94$
Salinas Valley	Métricas OA, AA e Kappa iguais ou superiores às metas	Comparação com as metas e com os valores reportados por Souza (2023)	$OA \geq 97\%$, $AA \geq 96\%$, $Kappa \geq 0{,}96$
University of Pavia	Métricas OA, AA e Kappa iguais ou superiores às metas	Comparação com as metas e com os valores reportados por Souza (2023)	$OA \geq 98\%$, $AA \geq 97\%$, $Kappa \geq 0{,}97$

3.6 ANÁLISE DE RISCOS

A análise de riscos identifica potenciais problemas que podem impactar o desenvolvimento e execução do projeto, estabelecendo estratégias de mitigação para cada cenário.

O Quadro 9 apresenta os principais riscos identificados, classificados quanto à probabilidade de ocorrência e ao impacto potencial no projeto. Nenhum risco foi classificado com probabilidade alta; o de maior impacto é o de o tempo de HPO exceder o tempo disponível, de probabilidade média e impacto alto.

Quadro 9 – Análise de riscos

Risco	Probab.	Impacto	Mitigação
Tempo de HPO excede o tempo disponível	Média	Alto	Reduzir espaço de busca e limitar número máximo de avaliações
Recursos computacionais insuficientes	Baixa	Médio	Utilizar Google Colab Pro e reduzir o tamanho do lote
Dificuldade de reproduzir baseline	Média	Médio	Contatar autores originais, ajustar hiperparâmetros cuidadosamente
Variabilidade alta nos resultados	Média	Médio	Aumentar o número de execuções e fixar as sementes aleatórias

4 DESENVOLVIMENTO

Este capítulo descreve a implementação do projeto do Capítulo 3: o ambiente e o código, o fluxo de execução, o paralelismo, a automação de hiperparâmetros, os experimentos e os desvios em relação ao projeto.

4.1 AMBIENTE E ORGANIZAÇÃO DO CÓDIGO

A implementação usa Python 3.12, PyTorch 2.13 com CUDA 13.0 (Paszke et al., 2019), scikit-learn para o PCA e as métricas, OpenCV com o módulo de contribuições para a segmentação e Optuna para a Otimização Bayesiana (Akiba et al., 2019), com as versões fixadas em um arquivo de requisitos versionado. Os experimentos rodaram em um computador pessoal com GPU NVIDIA RTX 5070 Laptop, 32 núcleos lógicos e 15,8 GB de RAM, sob WSL2. O notebook de Souza (2023), com estado global compartilhado entre células, foi reorganizado em um pacote Python, com um módulo por etapa do fluxo, um por pilar de otimização e um por experimento (01 a 08). O notebook apenas chama o pacote, e as variáveis de execução (componentes do PCA, parâmetros dos segmentadores, espaço de busca e cenas, sementes e orçamentos de cada experimento) ficam em um único arquivo de configuração versionado.

Os dados de entrada da rede são idênticos aos do notebook executado no Google Colab. O PCA usa o algoritmo das versões do scikit-learn anteriores à 1.5, já que o atual decide o sinal das componentes de outra forma, o que inverte um canal da composição RGB e muda a segmentação em Indian Pines e University of Pavia; a divisão repete as sementes e o sorteio do notebook, inclusive a segunda divisão, estratificada pelos rótulos em *one-hot*. Diferem do original o framework (PyTorch, e não TensorFlow/Keras), a arquitetura, que é a do projeto em todos os experimentos, exceto na comparação da linha de base, e o treino, com regra de parada e teto de épocas próprios (subseção 4.2.3). A execução é determinística: as sementes são fixadas, o cuDNN não escolhe algoritmos automaticamente, a precisão TF32 é desativada, e o OpenCV usa uma thread (seção 5.3).

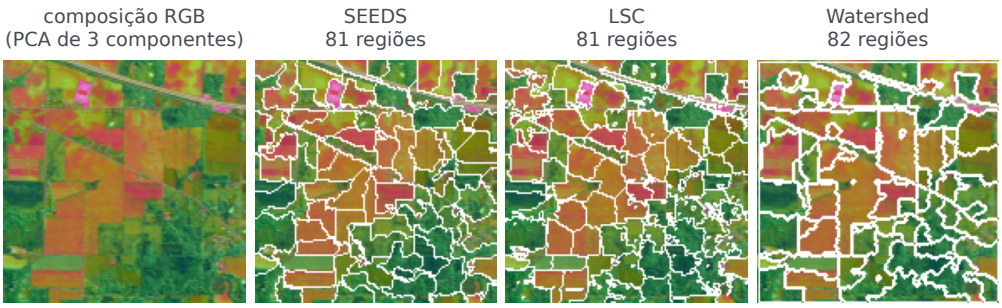
4.2 FLUXO DE EXECUÇÃO

Uma execução carrega o cubo (H, W, B) e a verdade terrestre; no ramo espectral, reduz as B bandas dos pixels rotulados a K_b componentes por PCA com branqueamento (30 em Indian Pines e 15 nas outras cenas); no ramo espacial, gera uma composição RGB de 8 bits com um PCA de 3 componentes da cena inteira e a segmenta em superpixels, cujo identificador é a segunda entrada da rede; divide as amostras; treina a CNN 1D; e calcula OA, AA e Kappa no teste. Os dois ramos são independentes até a fusão.

4.2.1 Segmentadores

SEEDS, LSC e Watershed são implementados sobre o OpenCV e devolvem o mesmo mapa de rótulos (H, W) (Figura 28). O SEEDS usa os parâmetros do notebook original, que resultam em 81 regiões em Indian Pines, 45 em University of Pavia e 96 em Salinas Valley. O LSC (tamanho de região e razão) e o Watershed (marcadores em grade regular, com as linhas entre bacias como uma região própria) foram calibrados para gerar o mesmo número de regiões em cada cena, para que a comparação reflita a forma das regiões, e não a sua quantidade. Para a busca, cada segmentador tem ainda duas variações, com cerca de 0,7 e de 1,2 a 1,4 vez as regiões do padrão.

Figura 28 – Composição RGB de Indian Pines e as regiões de cada segmentador, com os parâmetros padrão



Fonte: Elaborada pelo autor

4.2.2 Modelo e Fusão

A CNN 1D é a prevista no projeto (Figura 24 e Tabela 1), com os tamanhos de *kernel* do original e preenchimento que mantém o comprimento. A fusão é tardia, como no original: o identificador numérico do superpixel é concatenado ao vetor achatado das convoluções, em um passo próprio, que a fusão concorrente protege com uma trava. A CNN de Souza (2023) (16, 32, 64 e 64 filtros, sem preenchimento nem *pooling*, densas de 128 e 64 neurônios e 160.912 parâmetros em Indian Pines) aparece apenas na linha de base.

Tabela 1 – Camadas da CNN 1D do projeto com fusão, em Indian Pines ($K_b = 30$)

Camada	Saída (canais, comprimento)	Parâmetros
Conv1D 16, $k = 9$, ReLU, *max-pooling* 2	(16, 15)	160
Conv1D 32, $k = 3$, ReLU, *max-pooling* 2	(32, 7)	1.568
Conv1D 64, $k = 3$, ReLU, *max-pooling* 2	(64, 3)	6.208
Conv1D 128, $k = 3$, ReLU, achatamento	384	24.704
Fusão: concatenação do identificador	385	—
Densa 256, ReLU, *dropout*	256	98.816
Densa 128, ReLU, *dropout*	128	32.896
Densa de saída (*logits*)	16	2.064
Total		**166.416**

Fonte: Elaborada pelo autor. Em University of Pavia e Salinas Valley ($K_b = 15$), o total é de 99.977 e 100.880 parâmetros

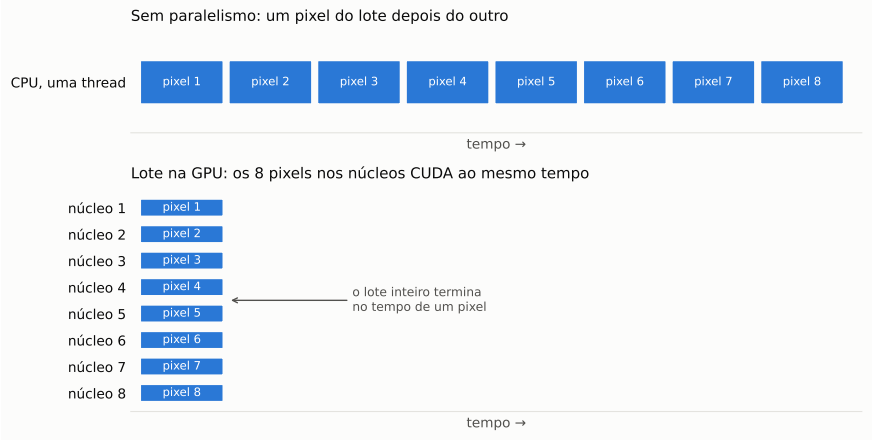
4.2.3 Treino e Divisão dos Dados

O treino usa a entropia cruzada e o Adam e para após 10 épocas sem que a acurácia de validação melhore 0,001, com teto de 200 épocas, restaurando os pesos da melhor época; o original parava pela perda de validação, com paciência de 5 e teto de 60. A troca faz a mesma métrica governar a parada e a seleção de hiperparâmetros. No experimento 04, o treino de cada *trial* termina também quando a validação atinge 0,95; nos experimentos 06 e 07, o teto é de 25 épocas, com lote 64. A divisão separa 85% das amostras para treino e divide o restante entre validação e teste, com semente fixa (42); a semente de cada execução muda só os pesos iniciais, a ordem dos lotes e o *dropout*. Só o experimento 04 usa 10% do treino, com validação e teste inteiros.

4.3 PARALELISMO

Antes das três estratégias, o treino já é paralelo dentro de cada operação: cada lote vai inteiro para a GPU, que calcula as convoluções e as camadas densas de todos os pixels do lote ao mesmo tempo (Figura 29). As durações nos esquemas são ilustrativas.

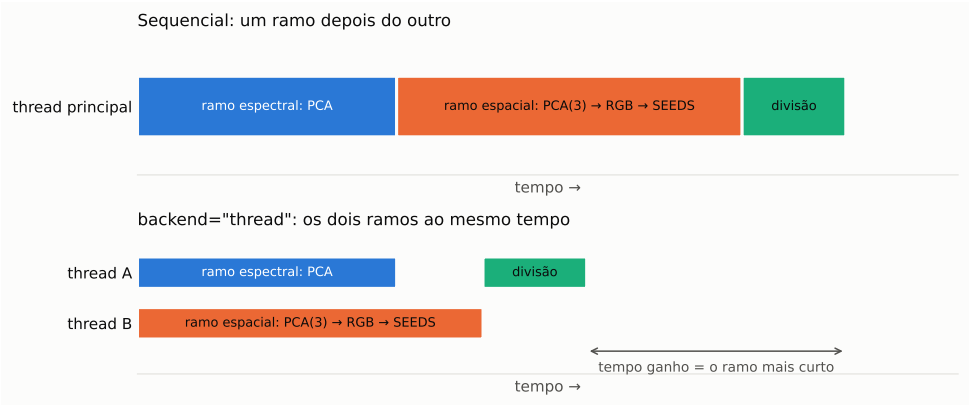
Figura 29 – Um lote processado pixel a pixel e na GPU



Fonte: Elaborada pelo autor. Esquema com 8 pixels; os lotes do trabalho têm de 32 a 128

A primeira estratégia executa os dois ramos do pré-processamento em duas threads. Elas avançam juntas mesmo com o GIL, pois o PCA e o segmentador são código nativo, que libera essa trava enquanto calcula (Figura 30).

Figura 30 – Os ramos do pré-processamento em sequência e em duas threads



Fonte: Elaborada pelo autor

A segunda divide a cena em N faixas horizontais com número semelhante de pixels rotulados, depois de segmentar a cena inteira uma única vez. No modo *independente*, cada partição treina e testa o seu modelo; no *replicado*, um modelo treinado com a cena inteira infere as partições. As partições executam na CPU, em sequência, em threads ou em processos, com uma thread do PyTorch cada. Os processos são criados por *spawn*, uma vez que o PyTorch interrompe com erro a cópia de processo feita depois de o cálculo de gradientes rodar em threads (PyTorch Contributors, 2025), e ficam em um pool reutilizável, aquecido fora da medição; um pool que não caberia na memória livre com 25% de folga, a 481 MB por processo, é recusado. A

Figura 31 compara os executores no modo independente, com o pool criado dentro da medição (frio) e antes dela (quente).

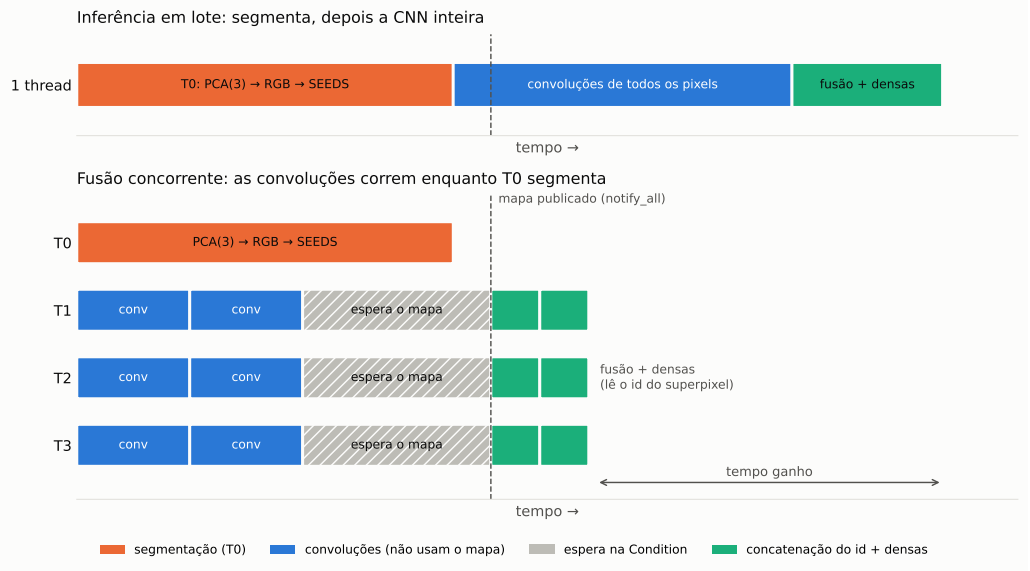
Figura 31 – A cena em quatro faixas e o treino delas em cada executor



Fonte: Elaborada pelo autor

A terceira, a fusão concorrente, realiza na inferência o esquema do projeto (Figura 26): uma thread T0 executa o ramo espacial e publica o mapa de segmentação, enquanto um pool de threads calcula as convoluções de blocos de 1, 64 ou 1.024 posições, lê o identificador de cada posição e faz a concatenação. Só a concatenação espera o mapa, por uma variável de condição (seção 2.5). A seção crítica tem duas extensões: **como no esquema**, em que T0 detém a trava durante a segmentação e cada tarefa durante a concatenação, e **mínima**, em que a trava só publica o mapa e lê os identificadores. Uma *variante desacoplada* enfileira todas as convoluções antes das fusões. Cada thread do pool fixa o número de threads do PyTorch, que de outro modo subiria às 32 do OpenMP e disputaria os mesmos núcleos. A Figura 32 mostra o ganho possível: o tempo total nunca fica abaixo do da segmentação, e o que se ganha é esconder as convoluções atrás dela.

Figura 32 – Inferência em lote e fusão concorrente



Fonte: Elaborada pelo autor

4.4 AUTOMAÇÃO DE HIPERPARÂMETROS

O espaço de busca tem sete hiperparâmetros com três valores cada, incluindo os da configuração original (Quadro 10): $3^7 = 2.187$ combinações por cena, de 15 a 23 h por semente em Indian Pines. Por isso, a **busca em grade** cobre o segmentador, a taxa de aprendizado e o *dropout*, os dois de maior efeito em buscas preliminares, com os demais no valor original (27 pontos), e a **Otimização Bayesiana** com processo gaussiano (Wu et al., 2019), implementada com o Optuna, recebe o mesmo orçamento de 27 *trials* no espaço inteiro. No amostrador, o segmentador e a sua configuração são categóricos, e os demais eixos, inteiros ordenados; os quatro primeiros *trials* são sorteados, com cada segmentador ao menos uma vez, e um candidato repetido recebe o valor já medido. Cada *trial* vale pela maior acurácia de validação, e o teste só é registrado.

Quadro 10 – Espaço de busca de hiperparâmetros

Hiperparâmetro	**Valores**	**Original**
Taxa de aprendizado	0,0003; 0,001; 0,003	0,001
Dropout	0,20; 0,35; 0,50	0,50
Weight decay	0; 10^{-5}; 10^{-4}	0
Tamanho do lote	32; 64; 128	32
Componentes do PCA	15; 30; 45 (Indian Pines) e 8; 15; 22	30 e 15
Segmentador	SEEDS; LSC; Watershed	SEEDS
Configuração do segmentador	padrão e duas variações	padrão

Fonte: Elaborado pelo autor

4.5 ORGANIZAÇÃO DOS EXPERIMENTOS

O plano de verificação (Quadro 4) foi desdobrado nos oito experimentos do Quadro 11.

Quadro 11 – Experimentos da campanha

Exp.	Objetivo	Cenas e sementes
01	Linha de base: configuração original com a CNN do projeto e com a de Souza (2023)	Três cenas; semente 42
02	Segmentadores com a CNN do projeto na configuração original	Três cenas; 5 sementes
03	Segmentadores com a CNN otimizada para cada um (16 *trials* de Otimização Bayesiana)	Três cenas; 5 sementes
04	Grade contra Otimização Bayesiana, 27 *trials* cada, até 200 épocas	Indian Pines; 3 sementes
05	Configuração otimizada no experimento 03 contra a original	Indian Pines; 5 sementes
06	Partição em 2 a 16 faixas: independente em processos e replicado em threads	Indian Pines; 3 sementes
07	Pool frio e quente de threads e de processos contra a execução sequencial	Indian Pines; semente 42
08	Fusão concorrente contra a execução em série	Três cenas; semente 42

Fonte: Elaborado pelo autor

4.6 DESVIOS EM RELAÇÃO AO PROJETO

A implementação seguiu o projeto quanto ao PyTorch, à arquitetura, aos três segmentadores, às duas estratégias de busca e à divisão dos dados, com os desvios abaixo.

- **Busca**: o Optuna executa as duas estratégias, visto que o seu processo gaussiano aceita eixos categóricos, como o segmentador; a grade cobre 27 pontos, e não os 2.187; a arquitetura e o Adam ficam fixos, e as épocas são decididas pela parada antecipada.
- **Fusão**: mantido o identificador numérico do original, no lugar da codificação *one-hot*, pois a fusão faz parte da configuração de referência; nenhuma configuração sem fusão é treinada, e a vantagem sobre a rede puramente espectral (RF03) é a reportada por Souza (2023).
- **Treino e ambiente**: parada pela acurácia de validação e teto de 200 épocas; GPU local, no lugar do Google Colab ou de HPC.
- **Campanha**: a otimização isolada de cada componente deu lugar à comparação dos segmentadores com a CNN fixa (02) e otimizada (03); os experimentos 04 a 07 rodam só em Indian Pines, a cena de menor acurácia; o 04 treina com 10% dos 85% de treino, pelo custo dos 162 *trials*; a linha de base usa uma semente; e os testes de significância deram lugar ao critério de duas vezes o desvio combinado.
- **Paralelismo**: threads e processos na CPU, no lugar da segmentação em processo e da CNN na GPU: a comparação entre executores exige o mesmo dispositivo.

5 RESULTADOS

Este capítulo apresenta o protocolo, a linha de base (experimento 01), os resultados de cada pilar, a verificação dos requisitos e a discussão.

5.1 PROTOCOLO DE EXECUÇÃO

Todos os experimentos usam o ambiente da seção 4.1, em modo determinístico, e a CNN do projeto (Tabela 1), salvo a comparação do experimento 01 com a CNN de Souza (2023). Quando não são objeto de busca, os hiperparâmetros são os do original: taxa de aprendizado 0,001, *dropout* 0,5, sem *weight decay*, lote 32 e 30 componentes em Indian Pines e 15 nas outras cenas. As acurácias são médias entre sementes (Quadro 11), com o desvio padrão populacional. Uma diferença entre duas configurações é afirmável quando excede duas vezes o desvio combinado, a raiz da soma dos quadrados dos desvios das duas.

5.2 LINHA DE BASE

A Tabela 2 compara os valores reportados por Souza (2023) com os do experimento 01, com a CNN do original e com a do projeto, ambas com os hiperparâmetros originais, o SEEDS, a regra de parada da subseção 4.2.3 e o treino completo. Com a CNN do original, os valores reportados são superiores por 1,82 ponto de OA em Indian Pines, 0,75 em University of Pavia e 0,14 em Salinas Valley, o que é coerente com o ajuste da parada por conjunto de dados relatado pelo autor, enquanto aqui a mesma regra vale para as três cenas. A CNN do projeto iguala a OA reportada em Indian Pines (0,9142) e fica 0,87 e 0,27 ponto abaixo nas outras cenas; com uma semente, essas diferenças não podem ser avaliadas pelo critério da seção 5.1.

Tabela 2 – Metas do projeto, valores reportados por Souza (2023) e valores do experimento 01

Cena	Origem	OA	AA	Kappa	Épocas
Indian Pines	Meta do projeto	0,95	0,93	0,94	—
	Reportado	0,9142	0,8956	0,9019	—
	CNN de Souza (2023)	0,8960	0,8717	0,8813	63 (53)
	CNN do projeto	0,9142	0,8625	0,9020	52 (42)
University of Pavia	Meta do projeto	0,98	0,97	0,97	—
	Reportado	0,9928	0,9866	0,9905	—
	CNN de Souza (2023)	0,9853	0,9781	0,9806	30 (20)
	CNN do projeto	0,9841	0,9771	0,9789	26 (16)
Salinas Valley	Meta do projeto	0,97	0,96	0,96	—
	Reportado	0,9965	0,9970	0,9962	—
	CNN de Souza (2023)	0,9951	0,9959	0,9945	25 (15)
	CNN do projeto	0,9938	0,9931	0,9931	29 (19)

Fonte: Elaborada pelo autor, com as metas do Quadro 8 e os valores reportados por Souza (2023). Experimento 01: semente 42. Épocas: treinadas e, entre parênteses, a época cujos pesos são restaurados; —: não se aplica ou não informado

A linha de base atinge as metas em University of Pavia e Salinas Valley e não as atinge em Indian Pines, cuja meta de OA está acima do próprio valor reportado. Em Indian Pines, a distância de 5,2 pontos entre OA e AA vem de uma classe rara: *Oats*, com 2 amostras de teste, não teve acerto e tira sozinha 6,25 pontos da AA; 56 dos 66 erros do teste ocorrem entre as classes de milho e de soja, cujos espectros são próximos.

5.3 ORGANIZAÇÃO DO CÓDIGO

A mesma configuração, com a mesma semente, produz as mesmas métricas: executadas de novo, as duas CNNs repetiram OA, AA, Kappa e número de épocas nas três cenas, e a CNN do projeto com o SEEDS e a semente 42 repetiu esses valores nos experimentos 02 e 05. Na partição, as execuções em sequência e em processos repetiram a OA em todas as repetições, e as 723 execuções da fusão concorrente, as classes da inferência em lote. A exceção são as threads da partição (seção 5.6). A organização tornou visíveis duas divergências que mudavam os dados, já corrigidas (seção 4.1), e dois defeitos.

Não determinismo do LSC. A implementação paralela do LSC no OpenCV desempata pela ordem das threads: com 32 threads, chamadas com a mesma imagem e os mesmos parâmetros diferem em 188 a 322 pixels em Indian Pines e em 142 a 330 em University of Pavia. Com o LSC no espaço de busca, a pontuação de um mesmo *trial* variava entre execuções, e a Otimização Bayesiana propagava essa variação para a busca. Com o OpenCV em uma thread, os rótulos se repetem, ao custo de deixar o LSC 2,4 vezes mais lento em Indian Pines (35,6 contra 14,8 ms) e 3,5 vezes em University of Pavia (289,7 contra 83,6 ms).

Número mínimo de componentes. As opções de PCA da busca usavam o piso da composição RGB (3 componentes), e não o da rede; com a CNN de Souza (2023), que exige ao menos 15 componentes, a opção de 8 em University of Pavia e Salinas Valley interrompia a campanha com um erro na convolução. O piso passou a ser derivado da arquitetura: 15 na CNN de Souza (2023) e 8 na do projeto.

5.4 SEGMENTADORES

Com o mesmo número de regiões, o SEEDS tem a maior pureza nas três cenas, e o Watershed é o mais rápido, de 28 a 62 vezes mais que o SEEDS (Tabela 3).

A Tabela 4 e a Figura 33 reúnem os três segmentadores com a CNN fixa na configuração original (experimento 02) e com a CNN otimizada para cada um (experimento 03), com 5 sementes e o treino completo. Com a CNN otimizada, o SEEDS supera o LSC de forma afirmável nas três cenas: por 1,72 ponto de OA em Indian Pines, acima do limiar de 1,52, por 0,77 em University of Pavia (limiar de 0,44) e por 0,32 em Salinas Valley (0,28), e o Kappa

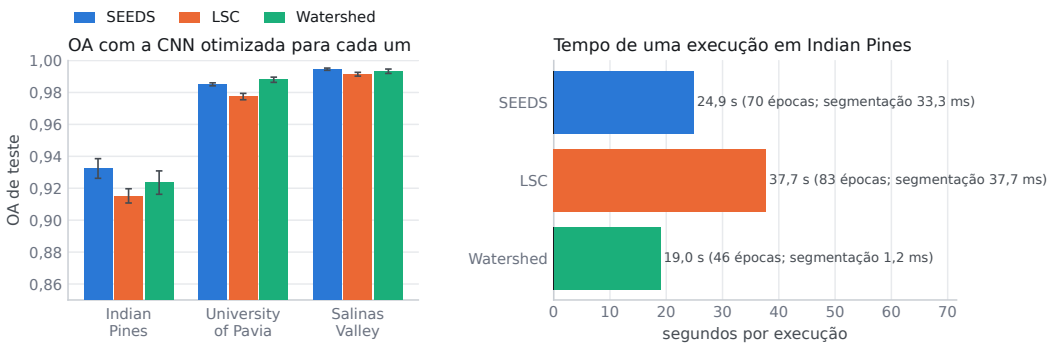
Tabela 3 – Estrutura das segmentações com os parâmetros padrão

Cena	Algoritmo	Regiões	Área média (px)	Pureza	Tempo (ms)
Indian Pines	SEEDS	81	259,6	0,8919	33,3
	LSC	81	259,6	0,8825	37,7
	Watershed	82	256,4	0,8330	1,2
University of Pavia	SEEDS	45	4.608,9	0,9272	599,1
	LSC	45	4.608,9	0,8194	280,2
	Watershed	46	4.508,7	0,7918	10,6
Salinas Valley	SEEDS	96	1.157,3	0,9886	343,1
	LSC	96	1.157,3	0,9241	151,0
	Watershed	97	1.145,4	0,9563	5,5

Fonte: Elaborada pelo autor. Área média: pixels da cena divididos pelo número de regiões. Pureza: fração dos pixels rotulados que pertencem à classe dominante da sua região. Tempo: mediana de 14 chamadas, com o OpenCV em uma thread, só da segmentação

segue a mesma ordem acima do limiar; a AA fica dentro do limiar nas três. Com a CNN fixa, a diferença só é afirmável em University of Pavia (1,23 ponto de OA, limiar de 0,59). Em Indian Pines, a busca deu aos dois a mesma CNN e o segmentador padrão (Tabela 5), e a ordem entre eles é a da pureza. O Watershed empata com o SEEDS em OA e Kappa nas três cenas e com as duas CNNs: com a otimizada, fica 0,89 ponto abaixo em Indian Pines (limiar de 1,92), 0,29 acima em University of Pavia (0,38), onde a AA é maior por 0,71 ponto, acima do limiar de 0,59, e 0,13 abaixo em Salinas Valley (0,31). Ele segmenta de 28 a 62 vezes mais rápido, mas, numa execução completa, a segmentação é menos de 0,2% do tempo em Indian Pines e menos de 0,7% nas outras cenas; o tempo varia com as épocas de cada configuração. Em University of Pavia e Salinas Valley, onde a OA com a CNN fixa já passa de 0,97, a otimização só supera a configuração original acima do limiar com o LSC em University of Pavia (0,65 ponto, limiar de 0,60), e a busca trocou os parâmetros do SEEDS para 202 superpixels, 250 iterações e *prior* 3.

Figura 33 – OA de teste com a CNN otimizada para cada segmentador (experimento 03) e tempo de uma execução em Indian Pines



Fonte: Elaborada pelo autor. Barras de erro: desvio padrão entre 5 sementes. Tempo: segmentação e treino de uma execução com a configuração vencedora de cada segmentador.

Tabela 4 – Segmentadores com a CNN fixa (experimento 02) e com a CNN otimizada para cada um (experimento 03)

Segmentador	OA, fixa	OA, otimizada	AA, otimizada	Kappa, otimizada
Indian Pines				
SEEDS	0,9069 ± 0,0081	0,9324 ± 0,0062	0,9361 ± 0,0162	0,9229 ± 0,0070
LSC	0,9038 ± 0,0060	0,9152 ± 0,0045	0,8895 ± 0,0181	0,9031 ± 0,0052
Watershed	0,8910 ± 0,0120	0,9235 ± 0,0073	0,9190 ± 0,0290	0,9128 ± 0,0083
University of Pavia				
SEEDS	0,9832 ± 0,0020	0,9851 ± 0,0009	0,9769 ± 0,0014	0,9802 ± 0,0012
LSC	0,9709 ± 0,0022	0,9774 ± 0,0020	0,9697 ± 0,0036	0,9700 ± 0,0027
Watershed	0,9834 ± 0,0024	0,9880 ± 0,0016	0,9840 ± 0,0026	0,9840 ± 0,0022
Salinas Valley				
SEEDS	0,9942 ± 0,0008	0,9946 ± 0,0007	0,9949 ± 0,0007	0,9940 ± 0,0008
LSC	0,9915 ± 0,0016	0,9914 ± 0,0012	0,9930 ± 0,0018	0,9905 ± 0,0013
Watershed	0,9928 ± 0,0008	0,9933 ± 0,0014	0,9944 ± 0,0016	0,9925 ± 0,0015

Fonte: Elaborada pelo autor. Média e desvio padrão entre 5 sementes, treino completo. CNN fixa: configuração original de Souza (2023).

Tabela 5 – Configurações vencedoras da busca do experimento 03

Segmentador	Taxa	*Dropout*	WD	Lote	PCA	Configuração	Validação	Épocas
Indian Pines								
SEEDS	0,0003	0,20	0	64	15	350 sp., 200 it., *prior* 2	0,9480	69,6
LSC	0,0003	0,20	0	64	15	16 px, razão 0,075	0,9389	82,6
Watershed	0,001	0,20	10^{-5}	64	15	116 marcadores	0,9389	46,4
University of Pavia								
SEEDS	0,0003	0,20	0	32	15	202 sp., 250 it., *prior* 3	0,9910	32,2
LSC	0,0003	0,20	0	64	15	65 px, razão 0,075	0,9863	38,2
Watershed	0,0003	0,20	10^{-4}	32	15	72 marcadores	0,9900	30,8
Salinas Valley								
SEEDS	0,0003	0,50	0	32	22	202 sp., 250 it., *prior* 3	0,9966	24,4
LSC	0,0003	0,20	0	32	15	34 px, razão 0,075	0,9946	26,8
Watershed	0,0003	0,35	0	32	15	136 marcadores	0,9953	27,6

Fonte: Elaborada pelo autor. Taxa: taxa de aprendizado; WD: *weight decay*; PCA: componentes; sp.: superpixels pedidos; it.: iterações. Validação: maior acurácia de validação entre os 16 *trials*. Épocas: média das épocas treinadas no retreino com 5 sementes.

5.5 AUTOMAÇÃO DE HIPERPARÂMETROS

A Tabela 6 compara a busca em grade e a Otimização Bayesiana com o mesmo orçamento de 27 *trials* de até 200 épocas e 3 sementes, em Indian Pines e com 10% do treino (experimento 04). A grade avalia os mesmos 27 pontos em cada semente, que muda só o treino; a bayesiana usa a semente também para sortear os *trials* iniciais. Cada *trial* para ainda se a validação atingir 0,95. Como o treino é reduzido, essas acurácias não se comparam às do treino completo. A tabela traz, por estratégia, a maior validação da busca, a OA de teste do vencedor e as épocas treinadas.

A bayesiana fica 1,95 ponto acima na validação e 1,48 na OA de teste do vencedor, dentro

Tabela 6 – Grade e Otimização Bayesiana com até 200 épocas por *trial* (experimento 04)

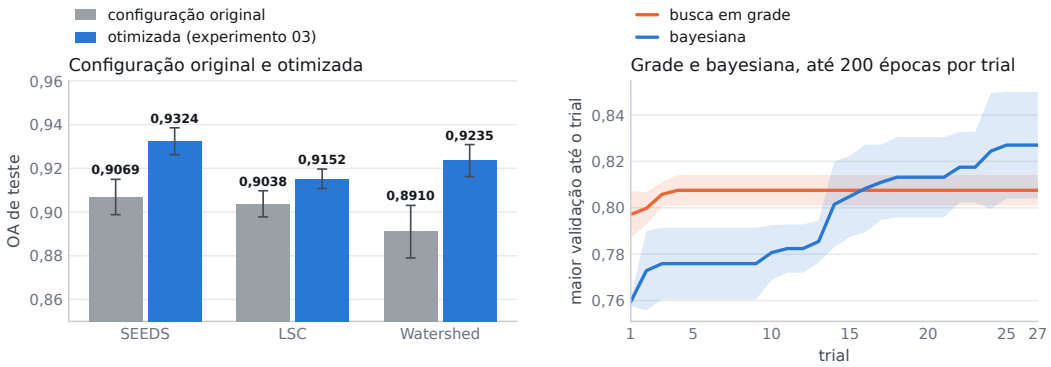
Estratégia	Maior validação	OA de teste do vencedor	Épocas treinadas
Grade	0,8075 ± 0,0066	0,7971 ± 0,0018	1.563
Bayesiana	0,8270 ± 0,0230	0,8119 ± 0,0198	1.833

Fonte: Elaborada pelo autor. Média e desvio padrão entre 3 sementes, 10% do treino, 27 *trials* por busca. Épocas treinadas: soma dos *trials* de uma busca, média entre as sementes

dos limiares de 4,8 e 4,0 pontos, e as estratégias empatam. Elas diferem no caminho (Figura 34, à direita): a grade chega ao seu melhor até o quarto *trial*, já que a ordem dos pontos começa pela menor taxa de aprendizado, e não melhora depois; a bayesiana parte de pontos sorteados e segue subindo até o 25º *trial*. Na grade, a validação média varia 2,4 pontos entre as taxas de aprendizado, 2,0 entre os valores de *dropout* e 1,6 entre os segmentadores, e as vencedoras da bayesiana usam todas *dropout* 0,2, o mesmo de duas das três vencedoras da grade.

O experimento 05 treina, com as mesmas 5 sementes em Indian Pines e o treino completo, a configuração original, de Souza (2023) com a CNN do projeto, e a vencedora do experimento 03 de cada segmentador, comparando as duas com o mesmo segmentador (Figura 34); os valores repetem as linhas de Indian Pines da Tabela 4. A vencedora com o SEEDS reduz a taxa de aprendizado para 0,0003 e o *dropout* para 0,2, dobra o lote e usa 15 componentes; como as mudanças vieram juntas, o efeito de cada uma não foi isolado. Com o SEEDS, a busca acrescenta 2,55 pontos de OA (de 0,9069 para 0,9324), acima do limiar de 2,03, e 2,92 de Kappa, acima de 2,31; com o Watershed, 3,25 pontos de OA, acima de 2,81. Com o LSC, o ganho de 1,14 fica dentro do limiar de 1,49. Na AA, nenhum ganho passa do limiar, que fica entre 5,4 e 7,6 pontos por causa das classes com 2 a 4 pixels de teste.

Figura 34 – OA de teste das configurações original e otimizada (experimento 05) e maior validação até cada *trial* com até 200 épocas (experimento 04), em Indian Pines



Fonte: Elaborada pelo autor. Barras de erro: desvio padrão entre 5 sementes; faixas: desvio padrão entre 3 sementes

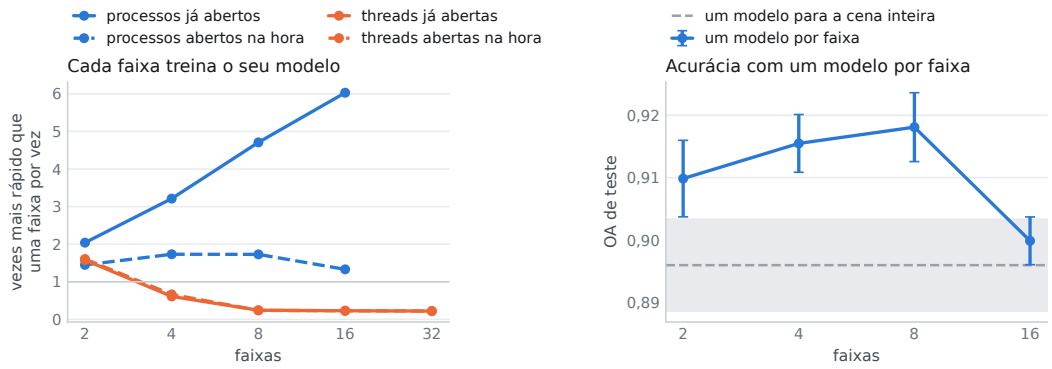
5.6 PARALELISMO

Os tempos são medianas das repetições. Nos ramos do pré-processamento, as duas threads levaram de 0,91 a 1,10 vez o tempo sequencial (de 0,97 a 2,18 s), com as faixas de 5 repetições sobrepostas: o ramo espacial domina, e a leitura e a divisão dos dados seguem em série. O pré-processamento é cerca de 4% de uma execução em Indian Pines, e o ganho precisa vir do treino.

Na partição da cena (Figura 35), com lote 64 e 25 épocas, o experimento 06 mede a OA com 3 sementes, e o 07, o *speedup* sobre a execução sequencial das mesmas partições, com a semente 42, 2 repetições com o pool frio e 3 com o quente; com $N = 32$, o pool de processos exigiria 19.240 MB e é recusado.

Com um modelo por faixa, a OA sobe 1,95 ponto com 4 faixas e 2,21 com 8 (0,9181 $\pm$ 0,0055) sobre o modelo único (0,8960 $\pm$ 0,0074), acima dos limiares de 1,74 e 1,84; com 2 e 16, fica dentro do limiar. O modo replicado repete a OA do modelo único. No tempo, só os processos com o pool quente escalam, de 2,06 vezes com 2 faixas a 6,06 com 16, contra um teto de 15,1 dado pela faixa mais lenta; com o pool frio, a partida dos processos, que importam o PyTorch, custa de 2,1 a 5,2 s e limita o ganho a 1,75 vez. As threads ganham 1,59 vez com 2 faixas e perdem para a execução sequencial a partir de 4, já que o laço de treino tem trechos em Python que disputam o GIL; no modo replicado, que só infere, em cerca de 15 ms, as threads ganham 1,30 vez com 2 faixas e perdem a partir de 4. Em sequência e em processos, a OA se repete exatamente; em threads, varia entre repetições (de 0,8934 a 0,9181 com 2 faixas), dado que elas compartilham o gerador aleatório global do PyTorch.

Figura 35 – *Speedup* do modo independente sobre a execução sequencial (experimento 07) e OA com um modelo por faixa (experimento 06), em Indian Pines



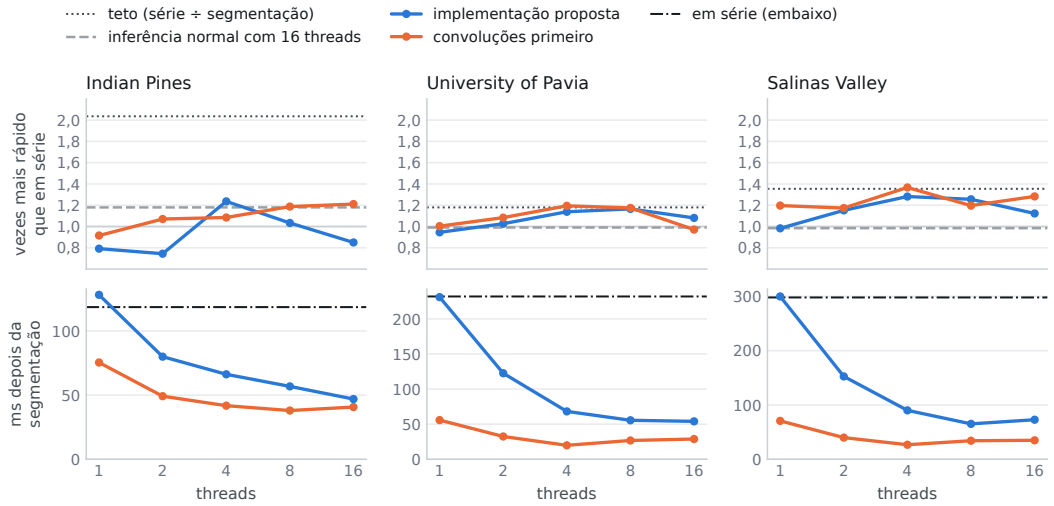
Fonte: Elaborada pelo autor. Processos e threads já abertos: pool quente; abertos na hora: pool frio. Direita: barras de erro e faixa cinza, desvio padrão entre 3 sementes

A fusão concorrente é medida na inferência de todos os pixels rotulados de cada cena,

com a CNN do projeto na configuração original e o SEEDS (experimento 08), contra a execução em série com o mesmo bloco, que segmenta e depois infere bloco a bloco em uma thread; o teto é a razão entre o tempo dela e o da segmentação: 2,04, 1,18 e 1,35. Em série, com bloco de 1.024, a inferência leva 253, 1.459 e 1.138 ms.

Com bloco de 1.024 e 4 ou mais threads, as duas formas chegam a 1,2 a 1,4 vez a execução em série (Figura 36), perto do teto em University of Pavia e Salinas Valley, mas oscilam com N e são da ordem da variação da própria segmentação, de 7% a 15% entre repetições. A medida que não depende dessa variação é o tempo que resta depois de T0 publicar o mapa: em série, restam 119, 232 e 298 ms; com bloco de 1.024 e 4 threads, a forma concorrente deixa 66, 68 e 90 ms, e a desacoplada, 42, 20 e 27 ms, escondendo atrás da segmentação de 45% a 71% e de 65% a 91% desse trabalho. Na concorrente, cada thread faz a convolução de um bloco e para à espera do mapa, e só 8% a 40% das convoluções terminam antes dele; na desacoplada, todas terminam. Com bloco 1, as 10.249 tarefas de um pixel tornam a inferência de 1,9 a 10 vezes mais lenta, e com bloco 64 a forma concorrente fica entre 0,74 e 0,87 vez a série a partir de 4 threads: coordenar as tarefas em Python, sob o GIL, custa mais que as convoluções. As 723 execuções reproduziram as classes da inferência em lote.

Figura 36 – Fusão concorrente contra a execução em série (experimento 08)



Fonte: Elaborada pelo autor. Bloco 1.024, seção crítica mínima (a seção como no esquema difere dela dentro da variação entre repetições), mediana de 5 repetições. Implementação proposta: forma concorrente; convoluções primeiro: forma desacoplada. Embaixo: tempo entre a publicação do mapa e o fim da inferência

5.7 VERIFICAÇÃO DOS REQUISITOS

O Quadro 12 reúne a evidência de cada requisito.

Quadro 12 – Evidência de cada requisito

Req.	Evidência
RF01	CNN 1D do projeto (Tabela 1); OA de 0,9142, 0,9841 e 0,9938 na linha de base
RF02	SEEDS, LSC e Watershed com o mesmo formato de saída e o mesmo número de regiões (Tabela 3)
RF03	Identificador concatenado ao vetor achatado: 385 entradas na primeira densa em Indian Pines e 129 nas outras cenas
RF04	Grade de 27 pontos e Otimização Bayesiana no espaço inteiro (Tabela 6 e Figura 34)
RF05	OA, AA, Kappa, tempos de segmentação, treino e inferência, parâmetros, memória por processo e *speedup*; o pico de memória não foi medido
RF06	Ramos do pré-processamento (0,91 a 1,10 vez), partição da cena (até 6,06 vezes) e fusão concorrente (até 1,37 vez) (seção 5.6)
RNF01	CNN do projeto derivada da de Souza (2023), com os mesmos *kernels* e a mesma fusão
RNF02	Implementação inteiramente em Python
RNF03	Treino em GPU local
RNF04	Fusão por concatenação do identificador ao vetor de características espectrais
RNF05	OA +2,55 e Kappa +2,92 pontos com o SEEDS em Indian Pines, acima do limiar; AA dentro do limiar
RNF06	Execuções repetidas idênticas, salvo em threads; desvio de OA entre sementes de 0,07 a 0,73 ponto
RNF07	*Speedup* maior que 1 na partição em processos e na fusão com bloco 1.024 e 4 threads

Fonte: Elaborado pelo autor

5.8 DISCUSSÃO DOS RESULTADOS

O ganho de cada otimização é limitado pela fração do trabalho que ela alcança: os ramos atuam sobre o pré-processamento e não ganharam, a fusão concorrente atua sobre a inferência, com teto dado pela segmentação, e só a partição alcança o treino, com o maior *speedup*. Em todas, o GIL e a partida dos *workers* pesaram mais que o cálculo. Medir na CPU é coerente com Souza (2023), em que a inferência com SEEDS foi mais rápida em uma CPU de 2 núcleos do que em uma GPU T4 (273 contra 426 ms em Indian Pines). Na acurácia, a busca pesou mais que o segmentador em Indian Pines, e nas outras cenas a configuração original já passa de 0,98; o Watershed empata com o SEEDS e é a alternativa quando a segmentação pesa, como na inferência.

6 CONSIDERAÇÕES FINAIS

Este trabalho partiu da CNN 1D com fusão de superpixels SEEDS de Souza (2023) e a otimizou em três pilares, segmentadores, hiperparâmetros e paralelismo, sobre um código reorganizado, medindo cada ganho contra a configuração de Souza (2023) com fusão SEEDS.

6.1 OBJETIVOS

O primeiro objetivo, uma implementação modular e reprodutível, foi atendido: o código é um pacote com um módulo por etapa e por pilar, os dados de entrada são idênticos aos do notebook original, e a mesma configuração produz sempre o mesmo resultado; a arquitetura é a do projeto, e o treino usa regra de parada e teto de épocas próprios. Com a CNN do projeto, a linha de base iguala a OA reportada em Indian Pines (0,9142) e fica 0,87 e 0,27 ponto abaixo nas outras cenas, em uma execução de cada lado. A organização revelou o não determinismo do LSC e o piso de componentes do PCA.

O segundo, explorar algoritmos de segmentação, foi atendido com SEEDS, LSC e Watershed calibrados para o mesmo número de regiões em cada cena. O SEEDS tem a maior pureza nas três cenas e, com a CNN otimizada para cada um, o SEEDS superou o LSC nas três cenas acima de duas vezes o desvio combinado (em Indian Pines, 0,9324 contra 0,9152) e empatou com o Watershed, que segmenta de 28 a 62 vezes mais rápido. O terceiro, o ajuste automático de hiperparâmetros, foi atendido com a busca em grade de 27 pontos e a Otimização Bayesiana com processo gaussiano; em Indian Pines, com 5 sementes, a busca elevou a OA com o SEEDS de 0,9069 para 0,9324 (+2,55 pontos) e a do Watershed em 3,25 pontos, ambas acima do limiar; com o LSC, o ganho de 1,14 ponto ficou dentro do ruído. Grade e bayesiana empataram dentro do limiar com o mesmo orçamento; a grade chegou ao seu melhor nos primeiros *trials*, e a bayesiana continuou melhorando até o fim da busca. O quarto, o processamento paralelo, foi implementado em três níveis: os ramos do pré-processamento em threads, que não ganharam (0,91 a 1,10 vez); a partição da cena em faixas, que chegou a 6,06 vezes a execução sequencial com 16 processos já abertos e, com 4 e 8 faixas, elevou a OA em cerca de 2 pontos sobre o modelo único; e a fusão concorrente na inferência, que chegou a 1,2 a 1,4 vez a execução em série e, na forma desacoplada, escondeu de 65% a 91% das convoluções atrás da segmentação, sempre com as classes da inferência em lote. O quinto, a análise comparativa, corresponde ao Capítulo 5. Com isso, o objetivo geral de analisar o impacto da segmentação, da parametrização automática e do processamento paralelo foi atendido: a parametrização trouxe o maior ganho de acurácia, em Indian Pines, a escolha do segmentador pesou menos que ela, e o paralelismo reduziu o tempo onde alcança o treino, mas encontrou no GIL e na partida dos processos seus principais limites.

6.2 HIPÓTESES

A primeira hipótese, de que os segmentadores apresentariam desempenhos variados e alternativas ao SEEDS para cenários específicos, se confirma nas três cenas: o Watershed empata em acurácia com o SEEDS e segmenta de 28 a 62 vezes mais rápido, o que o torna candidato quando a segmentação pesa no tempo total, e o LSC fica abaixo de ambos. A segunda, de que a otimização automática supera o ajuste manual de forma reprodutível, se confirma em Indian Pines com o SEEDS e o Watershed, mas não com o LSC; em University of Pavia e Salinas Valley, em que a configuração original já passa de 0,98, só o LSC em University of Pavia ganhou acima do limiar. A terceira, de que a otimização conjunta supera a isolada de cada componente, não foi testada como formulada, dado que a otimização isolada não foi executada. A quarta, de que a paralelização reduz o tempo de execução, se confirma para a partição em processos e para a fusão concorrente com blocos grandes, mas não para os ramos do pré-processamento nem para as threads com 4 ou mais faixas, que ficaram mais lentas que a execução sequencial.

6.3 LIMITAÇÕES E TRABALHOS FUTUROS

Só o experimento 04 usou 10% do treino, e a grade cobre só o segmentador, a taxa de aprendizado e o *dropout*. A busca de cada segmentador usou uma semente e 16 *trials*. A comparação das estratégias de busca, o ganho sobre a configuração original e a partição foram medidos apenas em Indian Pines, e a linha de base compara uma execução de cada lado. A comparação dos segmentadores vale para números de regiões próximos aos do SEEDS. O pico de memória não foi medido, e os tempos vêm de uma única máquina. Como trabalhos futuros, ficam: avaliar a codificação *one-hot* do identificador prevista no projeto; comparar os segmentadores com bem mais ou bem menos regiões; repetir a fusão concorrente e a partição em threads em uma versão do CPython sem a trava global (Gross, 2023); medir o método *forkserver* e a memória compartilhada entre processos (PyTorch Contributors, 2025); executar em paralelo os *trials* independentes da busca; e ampliar a campanha às três cenas, com a otimização isolada de cada componente e ablações da taxa de aprendizado, do *dropout*, do lote e do PCA.

REFERÊNCIAS

AACH, M. **Parallel and Scalable Hyperparameter Optimization for Distributed Deep Learning Methods on High-Performance Computing Systems**. 172 p. Tese (Ph.D. Dissertation) — University of Iceland, 2025. Disponível em: <https://doi.org/10.34734/FZJ-2025-02982>.

AHMAD, M. et al. A comprehensive survey for hyperspectral image classification: The evolution from conventional to transformers and mamba models. **Neurocomputing**, Elsevier BV, v. 644, p. 130428, set. 2025. ISSN 0925-2312. Disponível em: <http://dx.doi.org/10.1016/j.neucom.2025.130428>.

AKIBA, T. et al. Optuna: A next-generation hyperparameter optimization framework. In: **Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining**. [S.l.]: ACM, 2019. p. 2623–2631.

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: **Proceedings of the AFIPS Spring Joint Computer Conference**. New York: ACM, 1967. p. 483–485.

CHRONI, A. et al. Fusing multispectral and LiDAR data for CNN-based semantic segmentation in semi-arid mediterranean environments: Land cover classification and analysis. **Remote Sensing**, v. 16, n. 15, 2024. ISSN 2072-4292. Disponível em: <https://www.mdpi.com/2072-4292/16/15/2729>.

COHEN, J. A coefficient of agreement for nominal scales. **Educational and Psychological Measurement**, v. 20, n. 1, p. 37–46, 1960.

CONGALTON, R. G. A review of assessing the accuracy of classifications of remotely sensed data. **Remote Sensing of Environment**, v. 37, n. 1, p. 35–46, 1991.

COOK, S. **CUDA Programming: A Developer's Guide to Parallel Computing with GPUs**. 1. ed. Waltham, MA: Morgan Kaufmann, 2012. ISBN 978-0124159334.

DING, S. Q.; XIANG, C. Overfitting problem: a new perspective from the geometrical interpretation of MLP. In: ____. **Design and Application of Hybrid Intelligent Systems**. NLD: IOS Press, 2003. p. 50–57. ISBN 1586033948.

FARABET, C. et al. Learning hierarchical features for scene labeling. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 35, n. 8, p. 1915–1929, ago. 2013. ISSN 1939-3539.

FENG, X. et al. Computer vision algorithms and hardware implementations: A survey. **Integration**, v. 69, p. 309–320, 2019. ISSN 0167-9260. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167926019301762>.

GIL, A. C. **Como Elaborar Projetos de Pesquisa**. 4. ed. São Paulo: Atlas, 2002.

GONZALEZ, R.; WOODS, R. **Digital Image Processing**. Pearson, 2018. ISBN 9781292223049. Disponível em: <https://books.google.com.br/books?id=XmZvtAEACAAJ>.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>.

GORELICK, M.; OZSVALD, I. **High Performance Python: Practical Performant Programming for Humans**. 2. ed. [S.l.]: O'Reilly Media, 2020. ISBN 978-1492055020.

GROSS, S. **PEP 703 – Making the Global Interpreter Lock Optional in CPython**. 2023. Python Enhancement Proposals. Disponível em: <https://peps.python.org/pep-0703/>. Acesso em: 2 nov. 2026.

HOARE, C. A. R. Monitors: An operating system structuring concept. **Communications of the ACM**, v. 17, n. 10, p. 549–557, 1974.

IMANI, M.; GHASSEMIAN, H. An overview on spectral and spatial information fusion for hyperspectral image classification: Current trends and challenges. **Information Fusion**, v. 59, p. 59–83, 2020. ISSN 1566-2535. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1566253519307857>.

JOLLIFFE, I. T. **Principal Component Analysis**. 2. ed. New York: Springer, 2002.

JORDÀ, M.; VALERO-LARA, P.; PEÑA, A. J. Performance evaluation of cuDNN convolution algorithms on NVIDIA Volta GPUs. **IEEE Access**, IEEE, v. 7, p. 70461–70473, 2019.

JORDÀ, M.; VALERO-LARA, P.; PEÑA, A. J. cuConv: CUDA implementation of convolution for CNN inference. **Cluster Computing**, Springer, v. 25, n. 2, p. 1459–1473, 2022.

KINGMA, D. P.; BA, J. **Adam: A Method for Stochastic Optimization**. 2017. Disponível em: <https://arxiv.org/abs/1412.6980>.

KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. ImageNet classification with deep convolutional neural networks. In: PEREIRA, F. et al. (Ed.). **Advances in Neural Information Processing Systems**. Curran Associates, Inc., 2012. v. 25. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf>.

LABAN, N. et al. Reduced 3-D deep learning framework for hyperspectral image classification. In: ____. [S.l.: s.n.], 2020. p. 13–22. ISBN 978-3-030-14117-2.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, maio 2015. ISSN 1476-4687. Disponível em: <https://doi.org/10.1038/nature14539>.

LECUN, Y. et al. Gradient-based learning applied to document recognition. **Proceedings of the IEEE**, v. 86, n. 11, p. 2278–2324, nov. 1998. ISSN 1558-2256.

LI, C. et al. Directionally separable dilated CNN with hierarchical attention feature fusion for hyperspectral image classification. **International Journal of Remote Sensing**, v. 43, p. 812–840, fev. 2022.

LI, S. et al. Deep learning for hyperspectral image classification: An overview. **IEEE Transactions on Geoscience and Remote Sensing**, v. 57, p. 6690–6709, 2019. Disponível em: <https://api.semanticscholar.org/CorpusID:150101748>.

LI, Z.; CHEN, J. Superpixel segmentation using linear spectral clustering. In: **2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2015. p. 1356–1363.

LIASHCHYNSKYI, P.; LIASHCHYNSKYI, P. **Grid Search, Random Search, Genetic Algorithm: A Big Comparison for NAS**. 2019. Disponível em: <https://arxiv.org/abs/1912.06059>.

LU, B. et al. Recent advances of hyperspectral imaging technology and applications in agriculture. **Remote Sensing**, v. 12, n. 16, 2020. ISSN 2072-4292. Disponível em: <https://www.mdpi.com/2072-4292/12/16/2659>.

MITCHELL, T. **Machine Learning**. McGraw Hill, 2017. (McGraw Hill series in computer science). ISBN 9781259096952. Disponível em: <https://books.google.com.br/books?id=ifdcswEACAAJ>.

NAJMAN, L.; SCHMITT, M. Geodesic saliency of watershed contours and hierarchical segmentation. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 18, n. 12, p. 1163–1173, 1996.

NANDAN, A. D. M. Comparative analysis of TensorFlow and PyTorch for image classification using CNN with parallelism strategies. In: **2024 IEEE Silchar Subsection Conference (SILCON 2024)**. [S.l.: s.n.], 2024. p. 1–6.

NAQA, I. E.; MURPHY, M. J. What is machine learning? In: ____. **Machine Learning in Radiation Oncology: Theory and Applications**. Cham: Springer International Publishing, 2015. p. 3–11. ISBN 978-3-319-18305-3. Disponível em: <https://doi.org/10.1007/978-3-319-18305-3_1>.

NICKOLLS, J. et al. Scalable parallel programming with CUDA. **ACM Queue**, ACM, v. 6, n. 2, p. 40–53, 2008.

NOOR, M. H. M.; IGE, A. O. A survey on state-of-the-art deep learning applications and challenges. **Engineering Applications of Artificial Intelligence**, Elsevier BV, v. 159, p. 111225, nov. 2025. ISSN 0952-1976. Disponível em: <http://dx.doi.org/10.1016/j.engappai.2025.111225>.

PASZKE, A. et al. PyTorch: An imperative style, high-performance deep learning library. In: **Advances in Neural Information Processing Systems 32**. [S.l.: s.n.], 2019. p. 8024–8035.

PESARESI, M.; BENEDIKTSSON, J. A new approach for the morphological segmentation of high-resolution satellite imagery. **IEEE Transactions on Geoscience and Remote Sensing**, v. 39, n. 2, p. 309–320, fev. 2001. ISSN 1558-0644.

PyTorch Contributors. **Multiprocessing Best Practices**. 2025. Documentação do PyTorch. Disponível em: <https://pytorch.org/docs/stable/notes/multiprocessing.html>. Acesso em: 2 nov. 2026.

QI, W. et al. A spectral-spatial cascaded 3D convolutional neural network with a convolutional long short-term memory network for hyperspectral image classification. **Remote Sensing**, v. 11, p. 2363, out. 2019.

QIAN, S.-E. **Hyperspectral Satellites and System Design**. 1. ed. CRC Press, 2020. Disponível em: <https://doi.org/10.1201/9780429266201>.

SAMUEL, A. L. Some studies in machine learning using the game of checkers. **IBM Journal of Research and Development**, v. 3, n. 3, p. 210–229, 1959.

SERMANET, P. et al. **OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks**. 2014. Disponível em: <https://arxiv.org/abs/1312.6229>.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 10. ed. Hoboken: Wiley, 2018.

SIMONYAN, K.; ZISSERMAN, A. **Very Deep Convolutional Networks for Large-Scale Image Recognition**. 2015. Disponível em: <https://arxiv.org/abs/1409.1556>.

SOUZA, S. M. de. **Avaliação de Fusão de Dados Espaciais com Espectrais para Classificação de Imagens Hiperespectrais**. 99 p. Dissertação (Trabalho Técnico-científico de Conclusão de Curso (Graduação em Engenharia de Computação)) — Universidade do Vale do Itajaí, Escola Politécnica, Itajaí (SC), nov. 2023. Orientador: Felipe Viel, MSc.

SRIVASTAVA, N. et al. Dropout: a simple way to prevent neural networks from overfitting. **J. Mach. Learn. Res.**, JMLR.org, v. 15, n. 1, p. 1929–1958, jan. 2014. ISSN 1532-4435.

SZEGEDY, C. et al. **Going Deeper with Convolutions**. 2014. Disponível em: <https://arxiv.org/abs/1409.4842>.

VAN DEN BERGH, M. et al. **SEEDS: Superpixels Extracted via Energy-Driven Sampling**. 2013. Disponível em: <https://arxiv.org/abs/1309.3848>.

VIEL, F. **Acelerador em Hardware para Aumento da Resolução Espacial de Imagens Hiperespectrais em Microsatélites**. 115 p. Dissertação (Dissertação (Mestrado em Computação Aplicada)) — Universidade do Vale do Itajaí, Itajaí, SC, fev. 2019. Orientador: Cesar Albenes Zeferino.

VIEL, F. et al. Hyperspectral image classification: An analysis employing CNN, LSTM, Transformer, and attention mechanism. **IEEE Access**, v. 11, p. 24835–24850, 2023. ISSN 2169-3536.

WU, J. et al. Hyperparameter optimization for machine learning models based on Bayesian optimization. **Journal of Electronic Science and Technology**, v. 17, n. 1, p. 26–40, 2019. ISSN 1674-862X. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1674862X19300047>.